kernscope

A Technical Report

Static analysis, autotuning, verification, capture/replay, LLVM passes, and profiling for CUDA and HIP kernels

Six tools. One shared stack. One closed loop that leaves evidence another engineer can reproduce.

Technical report for the kernscope repository
31 October 2026

Measured evidence: AMD Radeon RX 7800 XT (gfx1101).
No NVIDIA runtime numbers are claimed.

Margins grown with `tikz-flora` *(MIT),* https://flora.alpacasoft.dev.

Abstract

kernscope is a C++20 toolchain for GPU kernels. Six tools profile a running binary, capture a hot launch, analyze the captured instruction set, generate or rewrite kernel variants, verify them before compilation, and benchmark the result. They share one library stack, one report schema, and one provenance discipline. The tools form a closed loop: the analyzer reads the captured kernel's ISA, the verifier gates the tuner's variants, and the profiler's hints cite the analyzer findings that explain them.

This report is the archival account of the project. It consolidates the architecture notes, the six per-tool references, the capture format and field mapping, the verifier soundness scope, the schema definitions, the reproducibility guide, the research briefs, and the committed evidence into one document. It describes the design and the implementation, then states what was measured on the local AMD Radeon RX 7800 XT (gfx1101), what remains projected from compile-only artifacts, and what was not tested. The measured results include a 153 s end-to-end demo against a 900 s budget, a bitwise capture replay, a 4-wide vectorization delta on layernorm, and machine peaks of 16.32 TFLOPS fp32, 591 GB/s DRAM read, and 9.7 TB/s LDS. No NVIDIA runtime numbers appear anywhere in the repository or in this report.

Preface: scope and evidence

This report replaces the pre-consolidation Markdown references and the earlier research report. It is written to be read on its own: every design claim, CLI contract, schema field, measured number, and known limitation is stated here or in an appendix. The editable Excalidraw diagrams remain in `docs/diagrams/` because they are also useful as standalone figures.

Three rules govern the evidence in this document. First, a number is either *measured* on the local gfx1101 or *projected* from static analysis or compile-only artifacts, and the report says which. Second, an NVIDIA runtime path is owner-gated and is not claimed until the owner produces evidence; the repository commits none at the time of writing. Third, a statement that cannot be checked against the tree, a committed artifact, or a cited source is flagged as a limitation instead of being smoothed over.

The report is organized in three parts. Part I describes the system, the hardware model, and the shared libraries. Part II documents the six tools and the benchmark/dashboard layer. Part III covers testing, reproducibility, evaluation, prior art, limitations, and future work. The appendices hold the reference material that would otherwise be scattered across per-tool notes: CLI synopses, check and rule tables, search spaces, bundle fields, report fields, a consolidation map, and a glossary.

Contents

Preface: scope and evidence ii

List of figures vii

List of tables viii

I System and architecture 1

1 Introduction 2
- 1.1 The problem . . . 2
- 1.2 What kernscope is . . . 2
- 1.3 Contributions . . . 2
- 1.4 Design principles . . . 3
- 1.5 Reading guide . . . 3

2 System architecture 3
- 2.1 Six tools at a glance . . . 3
- 2.2 The closed loop . . . 3
- 2.3 The shared stack . . . 5
- 2.4 Evidence flow . . . 5
- 2.5 Build and repository shape . . . 7

3 Hardware and execution model 7
- 3.1 The local live path . . . 7
- 3.2 CUDA is compile-only here . . . 7
- 3.3 The rental gate . . . 8
- 3.4 Capture mechanisms . . . 8
- 3.5 The CUDA 13 migration surface . . . 8
- 3.6 What the hardware split implies . . . 10

4 The shared libraries 10
- 4.1 Architecture database and record store . . . 10
- 4.2 PTX and AMDGPU parsers . . . 10
- 4.3 Occupancy and roofline . . . 10
- 4.4 Benchmark harness and runtime shim . . . 11
- 4.5 Report engine and provenance . . . 11
- 4.6 Capture backends . . . 11
- 4.7 Dependency rules . . . 11

II The tools **12**

5 C1: static analyzer **13**
5.1 Inputs, CLI, and precedence . . . 13
5.2 The ten analyses . . . 13
5.3 Suggestion rules . . . 13
5.4 Report and golden result . . . 13

6 C2: correctness-gated autotuner **15**
6.1 Families and search spaces . . . 15
6.2 The pipeline . . . 15
6.3 Outputs and reproducibility . . . 17
6.4 Capture-tune mode . . . 17
6.5 Measured evidence . . . 17

7 C3: static verifier **18**
7.1 Ingestion and launch tiers . . . 18
7.2 The eight check classes . . . 19
7.3 Verdict semantics and approved IR . . . 19
7.4 Soundness scope . . . 19
7.5 Corpus and evidence . . . 20

8 C4: capture and replay **20**
8.1 Mechanisms . . . 20
8.2 Bundle format v1 . . . 20
8.3 Replay, validation, and benchmark . . . 21
8.4 Reproducer and size governance . . . 21
8.5 CUDA/HIP field mapping . . . 21
8.6 Measured evidence . . . 22

9 C5: LLVM passes **22**
9.1 Plugin structure . . . 22
9.2 The passes . . . 22
9.3 The delta report . . . 23

10 C6: profiler **23**
10.1 Pipeline . . . 23
10.2 Diagnostics . . . 23
10.3 Measured evidence . . . 24

11 Bench, dashboard, and the evidence layer **24**
11.1 Benchmark methodology . . . 24

11.2 Measured machine peaks . . . 24
11.3 Dashboard sections . . . 24
11.4 What the provenance does and does not enforce . . . 25

III Evidence and operation **26**

12 Testing, CI, and reproducibility **27**
12.1 The four test layers . . . 27
12.2 The CI gate . . . 27
12.3 The master reproducibility script . . . 27
12.4 Owner-gated NVIDIA runs . . . 28
12.5 Secrets and sensitive data . . . 28

13 Evaluation **28**
13.1 The demo run . . . 28
13.2 Analyzer and verifier evidence . . . 29
13.3 Tuner and capture evidence . . . 29
13.4 Pass and profile evidence . . . 29
13.5 Threats to validity . . . 29

14 Context and prior art **30**
14.1 Performance models and static analysis . . . 30
14.2 Autotuning and tensor compilation . . . 30
14.3 GPU verification . . . 30
14.4 Capture and replay . . . 31
14.5 LLVM passes and profiling . . . 31

15 Limitations and future work **31**
15.1 Limitations . . . 31
15.2 Future work . . . 32

16 Conclusion **32**

A CLI reference **32**

B Verifier verdicts and engine ids **34**

C Capture bundle fields **34**

D Report schema and provenance **35**

E Consolidation map **35**

F Glossary

List of Figures

1 The kernscope loop. The forward path is profile → capture → analyze → transform/generate → verify → benchmark; the dashed edge is the delta report that starts the next pass. . . . 4

2 The shared stack. Six tools sit over four library groups and the LLVM/CUDA/ROCm toolchain. Each tool links only what it needs; no tool links another tool. . . . 5

3 Evidence flow. Tools emit report models through `libs/report`; the runtime artifacts are aggregated by the dashboard and frozen in this report. The provenance gate is a convention plus a separate validator, not a schema-enforced property. . . . 6

4 Three execution venues. The AMD column is live and measured, the NVIDIA compile column is projected, and the rental column is owner-gated and currently has no committed numbers. . . . 8

5 The tuner pipeline. The template path searches source variants; the capture path searches launch configurations over a captured binary. Both use the same benchmark and selection rules. . . . 16

6 The verifier pipeline. The launch tiers feed the diagnostic engine; the check registry produces per-class verdicts; approved IR is written only when no error-class verdict remains. . . . 18

7 Bundle v1. The manifest names the vendor and mechanism; the event log records API calls, allocations, launches, copies, memsets, streams, modules, and parameter metadata; the blob index carries a SHA-256 and a patch table for every snapshot. . . . 20

List of Tables

1 The six tools. kernscope-bench and kernscope-dashboard are supporting layers. . . . 4

2 Direct CMake dependencies of the tool targets. “LLVM only” means the tool does not link a `libs/` target. . . . 6

3 Measured gfx1101 peaks. All rows are `provenance.mode=measured`; the ridge point used by the tuner and profiler is $16.32 \times 10^{12} / 591 \times 10^{9} = 27.61$ FLOP/byte. . . . 7

4 Capture mechanisms and their local status. The NVIDIA paths never run live on this machine. . . . 9

5 The analyzer’s ten analyses. The occupancy model is the min-wins formula from the occupancy library; the latency estimate is explicitly approximate. 14

6 Suggestion rules. The priority order is the table order. The engine drops instruction-mix informational findings so a suggestion is tied to an actionable warning. . . . 15

7 Tuner families and their search spaces. . . . 15

8 The eight check classes and their stable IDs. Engine-degraded verdicts use IDs at or above 100: symbolic dims (100), SCEV reconstruction failed (101), and budget exceeded (102). . . . 19

9 The pass modules. The coalescing analysis is shared with the analyzer's 128-byte-sector model so before/after numbers use the same definition. . . 22
10 Dashboard sections and their provenance modes. The mode is a convention recorded by the aggregator; the report schema does not require it. 24
11 The CI stages. The count guard is non-decreasing, not exact: adding tests is always green, and removing a registration below the floor fails. 27
12 The five demo stages. The report's cross-tool block links the stage-3 analyzer finding ids to the stage-1 profiler hint. 28
14 The verdict vocabulary. SAFE is the only approval-class verdict; engine ids are degraded outcomes, not proofs. 34
15 Bundle v1 required fields. All 64-bit values are lowercase 0x-prefixed hexadecimal strings; small counters and dimensions are JSON numbers. Every blob hash is recomputed on read. 34
16 The report schema and the provenance convention that sits outside it. The distinction is deliberate: the schema validates shape, and the dashboard validator checks provenance. 35
17 Material folded into this report. The editable Excalidraw diagrams remain in `docs/diagrams/`; the test-critical before.ll inputs and the peak JSONL are retained under `tests/` because tests read them. 36

Part I

System and architecture

1 Introduction

1.1 The problem

GPU kernel work is split across tools that do not share identifiers. A vendor profiler says a kernel is hot; a disassembler shows the ISA; a template generator produces variants; a verifier checks a source file; a benchmark harness times a binary. Each tool re-derives the same architecture constants (register file, shared-memory budget, occupancy limits), each writes a different report, and none can show that its conclusion rests on the same launch the previous tool observed. The result is a pile of local screenshots and hand-copied numbers, not a reproducible evidence chain.

kernscope replaces the pile with one loop over real artifacts. The profiler ranks kernels from a running binary. Capture records the hot launch into a bundle that replays bitwise. The analyzer reads the captured kernel's disassembled ISA. The tuner generates variants, the verifier gates them before compilation, and the benchmark harness times only variants that passed the correctness gate. Each stage writes a report with a schema version and a provenance mode, so a later reader can tell a live measurement from a static projection.

1.2 What kernscope is

kernscope is a C++20 toolchain with six user-facing tools:

C1 `kernscope-analyze`
Static analysis of PTX or AMDGPU assembly: ten analyses plus a suggestion engine.

C2 `kernscope-tune`
Correctness-gated search over template kernel families, with roofline and occupancy evidence.

C3 `kernscope-verify`
Eight-class static verifier over LLVM IR, with an approved-IR handoff.

C4 `kernscope-capture`
Kernel capture and bitwise replay, plus standalone reproducer generation.

C5 `kernscope-pass`
Out-of-tree LLVM 21 pass plugin and the `kernscope-pass-report` delta CLI.

C6 `kernscope-profile`
Kernel-launch tracing, hot-kernel ranking, and compiler-aware "why slow" diagnostics.

The six tools live in one build and link the shared libraries they need. `kernscope-bench` is the peak and replay benchmark harness; `kernscope-dashboard` is a Python aggregator. The report treats those two as supporting layers, not as a seventh and eighth tool.

1.3 Contributions

1. A closed loop across six independently executable tools, wired by `scripts/demo.sh` and recorded in one cross-tool report.
2. A shared library stack with an architecture database, PTX and AMDGPU parsers, occupancy and roofline engines, a timing harness, a report engine, and capture backends.

3. A dual-path hardware strategy: every NVIDIA-runtime capability has a compile-only or mock implementation that runs on the AMD machine, while the HIP equivalent runs live on gfx1101.
4. A provenance discipline: per-module `provenance.mode`, a report schema, owner-gated rentals, and no vendored NVIDIA headers.
5. A four-layer test convention (ctest, lit, bf16 goldens, generated-code compile checks) with a non-decreasing count guard.
6. A measured evidence base from one machine, with the limits of that base stated next to the numbers.

1.4 Design principles

One implementation per fact. Architecture constants live in `libs/arch`; occupancy and roofline formulas live once in `libs/occupancy` and `libs/roofline`; the report schema and provenance writer live in `libs/report`; the timing method lives in `libs/bench`. A tool links what it needs and does not re-implement it.

Evidence over self-report. The analyzer runs on the captured ISA, not on a hand-picked example. The verifier gates tuner variants before compilation. The profiler hint carries the analyzer finding identifiers that explain it. The pass delta is measured with the analyzer's own metrics. A tool that only reports on itself is not used as evidence for another.

Two implementations, one contract. Every NVIDIA-runtime capability is paired with a HIP implementation on the local GPU, or with a mock/compile-only path. The two implementations share the same bundle, record, and report contracts, so the local machine exercises the code even when it cannot run the vendor runtime.

Provenance is part of the data. Reports carry `schema_version` and a per-module provenance object. A projected number is never placed in a measured slot. The rental path is owner-gated and not auto-triggered. The cost of this discipline is some missing evidence; the benefit is that every present number has a defined meaning.

1.5 Reading guide

Section 2 gives the system picture and the three architecture figures. Section 3 describes where code runs and how the CUDA-13 surface is isolated. Section 4 documents the shared libraries. Section 5 through Section 11 document the tools, their CLIs, and their evidence. Section 12 covers tests and reproducibility, Section 13 evaluates the measured results, Section 14 positions the work, and Sections 15 and 15.2 state what remains open. The appendices are the reference layer.

2 System architecture

2.1 Six tools at a glance

2.2 The closed loop

The loop is a convention over files and executables, not one monolithic binary. `scripts/demo.sh` runs five observable stages on the local gfx1101: profile, cap-

ID	Tool	Role
C1	`kernscope-analyze`	Ten static analyses of PTX or AMDGPU ISA, plus suggestions.
C2	`kernscope-tune`	Template search over six families, correctness-gated and roofline-aware.
C3	`kernscope-verify`	Eight IR-level checks; approved IR only when every verdict is `SAFE`.
C4	`kernscope-capture`	Record, replay, validate, benchmark, and emit a standalone reproducer.
C5	`kernscope-pass-report`	Out-of-tree LLVM 21 passes and analyzer-measured before/after deltas.
C6	`kernscope-profile`	Trace, enrich, rank, and diagnose kernels from a running binary.

Table 1. The six tools. kernscope-bench and kernscope-dashboard are supporting layers.

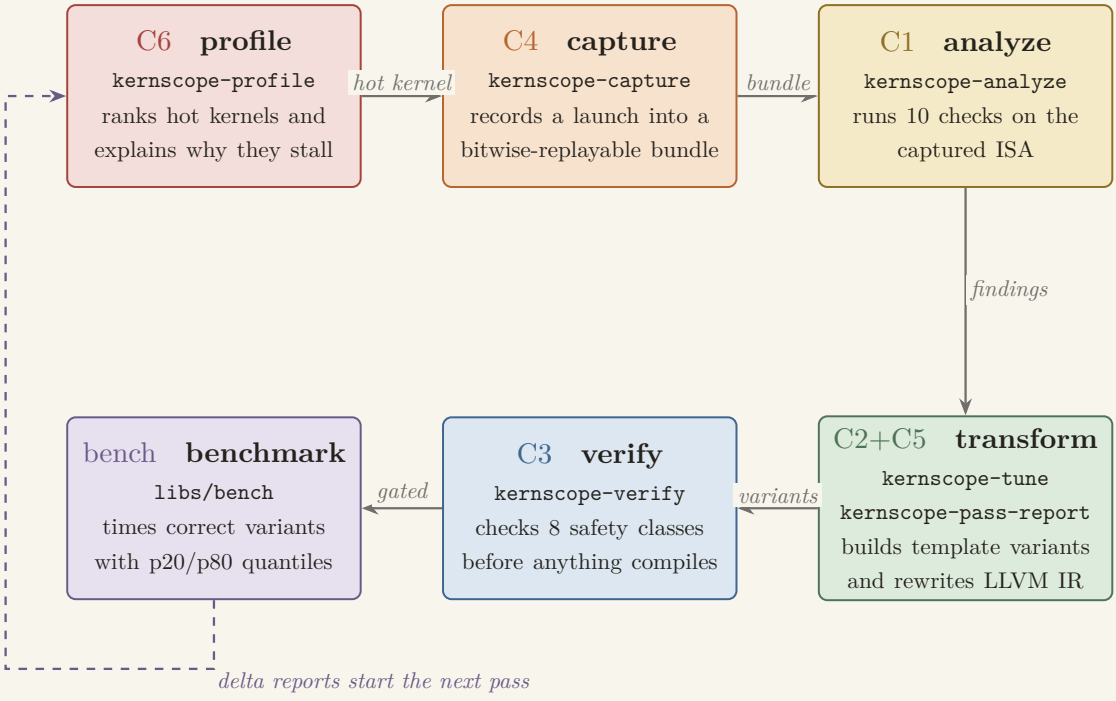


Figure 1. The kernscope loop. The forward path is profile → capture → analyze → transform/generate → verify → benchmark; the dashed edge is the delta report that starts the next pass.

ture, analyze, capture-tune, and verifier-gated tune plus a pass delta. It writes `reports/demo/report.json` and a self-contained HTML page, and it aborts early with a well-formed all-skipped report when `/dev/kfd` is absent or `KERNSCOPE_NO_GPU=1`. The committed run completed in 153 s against a 900 s budget.

Six cross-tool edges make the loop more than a pipeline:

- **profile → capture.** The profiler ranks hot kernels; capture records the hot one.
- **capture → analyze.** The analyzer runs on the captured kernel's disassembled ISA, not on a hand-picked example.
- **capture → tune.** `kernscope-tune --from-capture` searches launch configurations over the bundle and validates each candidate bitwise.
- **tune → verify.** `kernscope-tune --verify` gates variants before they are compiled.
- **pass → analyze.** `kernscope-pass-report --report-pipeline` measures before/after deltas with the analyzer's own metrics.
- **profile ↔ analyze.** The aggregated demo report links the profiler hint to the analyzer finding identifiers.

2.3 The shared stack

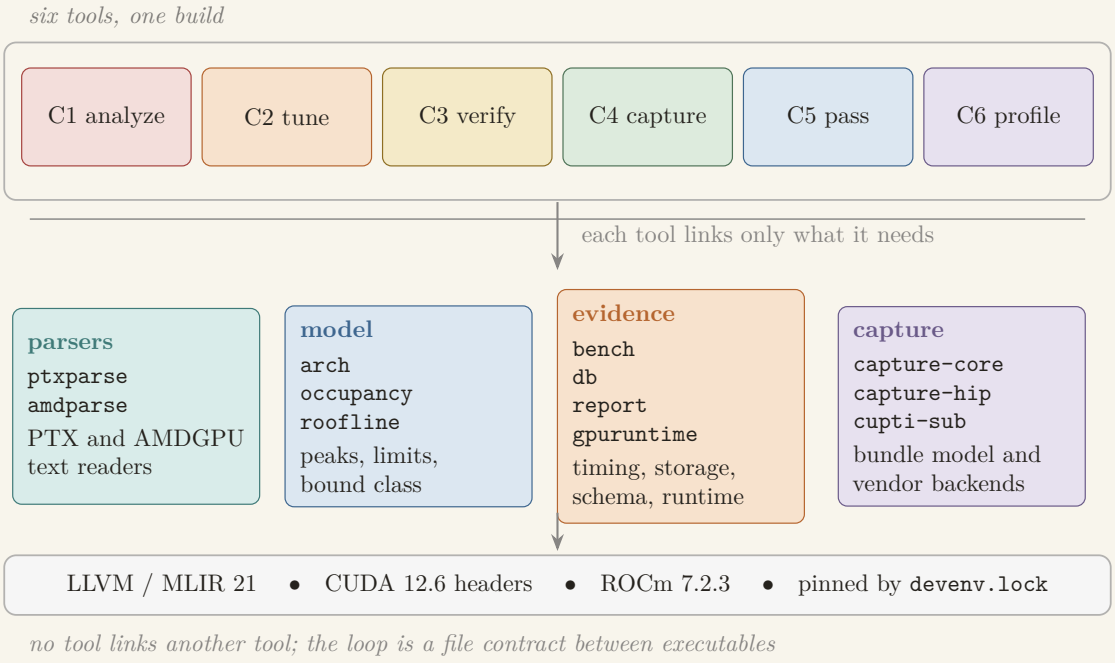


Figure 2. The shared stack. Six tools sit over four library groups and the LLVM/CUDA/ROCm toolchain. Each tool links only what it needs; no tool links another tool.

The stack has twelve libraries in four functional groups. *Parsers*: `ptxparse`, `amdparse`. *Model*: `arch`, `occupancy`, `roofline`. *Evidence*: `bench`, `db`, `report`, `gpuruntime`. *Capture*: `capture-core`, `capture-hip`, `cupti-sub`. The exact link graph, as built by CMake rather than as sketched in the older notes, is in Table 2.

2.4 Evidence flow

A tool report is a JSON document with a schema version and a provenance object. The demo aggregator combines the five stages into one model and keeps cross-tool links: the

Target	Direct dependencies
`kernscope-analyze`	`kernscope::analyze`, `kernscope::db`; the analyze library links `ptxparse`, `amdparse`, `arch`, `occupancy`, `report`.
`kernscope-tune`	`tune_driver`, `capture_tune`, `report`; the driver links the three generators, `arch`, `occupancy`, `roofline`, `bench`, `db`, `gpuruntime`.
`kernscope-verify`	LLVM only.
`kernscope-capture`	`capture_core`, `db`, `bench`, `gpuruntime`, `arch`; the HIP and CUPTI capture libraries are loaded as `LD_PRELOAD` shared objects, not linked into the CLI.
`kernscope-pass-report`	LLVM; the pass plugin is a separate `MODULE`.
`kernscope-profile`	`profile_diag`, `profile_enrich`, `roofline`, `report`, `db`; `profile_enrich` links `occupancy`; the CUPTI and rocprofiler tracers are separate libraries.
`kernscope-bench`	`bench`, `db`, `gpuruntime`, `arch`.

Table 2. Direct CMake dependencies of the tool targets. "LLVM only" means the tool does not link a `libs/` target.

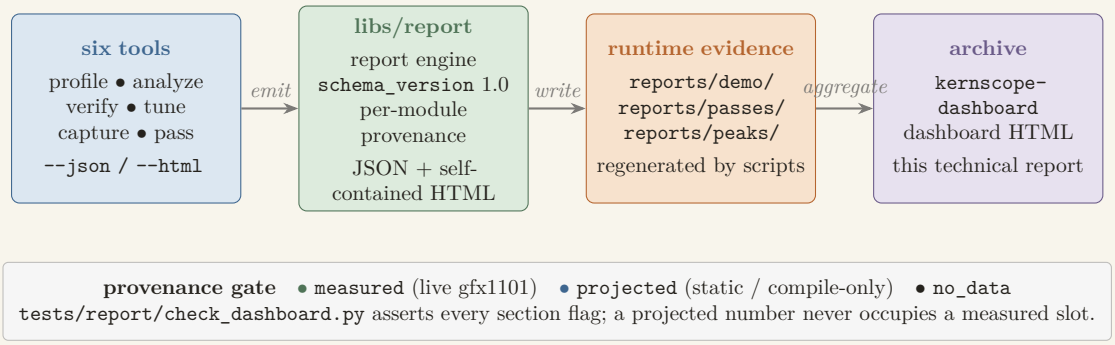


Figure 3. Evidence flow. Tools emit report models through `libs/report`; the runtime artifacts are aggregated by the dashboard and frozen in this report. The provenance gate is a convention plus a separate validator, not a schema-enforced property.

profiler hint carries the analyzer finding identifiers and the stage-3 evidence path. The dashboard has six sections (`analyzer`, `tune`, `verify`, `capture`, `profile`, `pass`), each with `provenance.mode` and a `measured` boolean. Section 11.4 states exactly what the schemas require and where the provenance convention lives outside them.

2.5 Build and repository shape

The project is a C++20 CMake superbuild. `CMakeLists.txt` requires LLVM 21.1 and treats MLIR as optional; it applies `-Wall -Wextra -Werror` globally. `scripts/configure.sh` resolves `LLVM_DIR` and `MLIR_DIR` from the nix store when the build is not already inside `devenv shell`. `devenv.nix` supplies LLVM/MLIR 21, the CUDA 12.6 redistributables, ROCm 7.2.3, gtest, lit, CMake, Ninja, and a Python with NumPy and pytest. The build registers the libraries first, then the tools, then the lit suite, so no tool can depend downward on another tool.

The repository keeps the source and the tests; the runtime evidence directory is generated. This report is the committed archive of the evidence that was present at consolidation time. Appendix E records what was folded here and what remains as a separate diagram asset.

3 Hardware and execution model

3.1 The local live path

The development machine has one AMD Radeon RX 7800 XT (gfx1101, RDNA 3) and no NVIDIA GPU. HIP code compiles with `hipcc` and runs live; `/dev/kfd` is the runtime gate. `libs/arch` holds the measured machine peaks, the per-architecture limits, and the occupancy constants. The peaks used by the roofline engine are shown in Table 3; they were measured with `kernscope-bench` on 2026-08-14 with clocks recorded but not locked.

Family	Microbenchmark	Median	Derived peak
`peak_flops`	register-resident FFMA loop	13.51 ms	16.32 TFLOPS fp32
`peak_bw_read`	512 MiB DRAM read	0.907 ms	591 GB/s
`peak_bw_d2d`	1 GiB device-to-device	2.26–2.30 ms	467–475 GB/s
`peak_lds`	LDS read + write	10.14–10.32 ms	9.7 TB/s

Table 3. Measured gfx1101 peaks. All rows are `provenance.mode=measured`; the ridge point used by the tuner and profiler is $16.32 \times 10^{12} / 591 \times 10^{9} = 27.61$ FLOP/byte.

3.2 CUDA is compile-only here

The CUDA path compiles kernels for sm_80 and sm_90 with `nvcc` against the CUDA 12.6 redistributables from the nix store. The resulting PTX and cubins are analyzed and compiled, but no CUDA runtime executes on the machine. `libs/gpuruntime` provides a mock backend for the capture shim, and `libs/cupti-sub` compiles and unit-tests the CUPTI subscription against synthetic activity records. The repository commits no NVIDIA headers: the nix store resolves `##include <cuda_occupancy.h>`-style names at build time, and `scripts/ci.sh` guards both filenames and content.

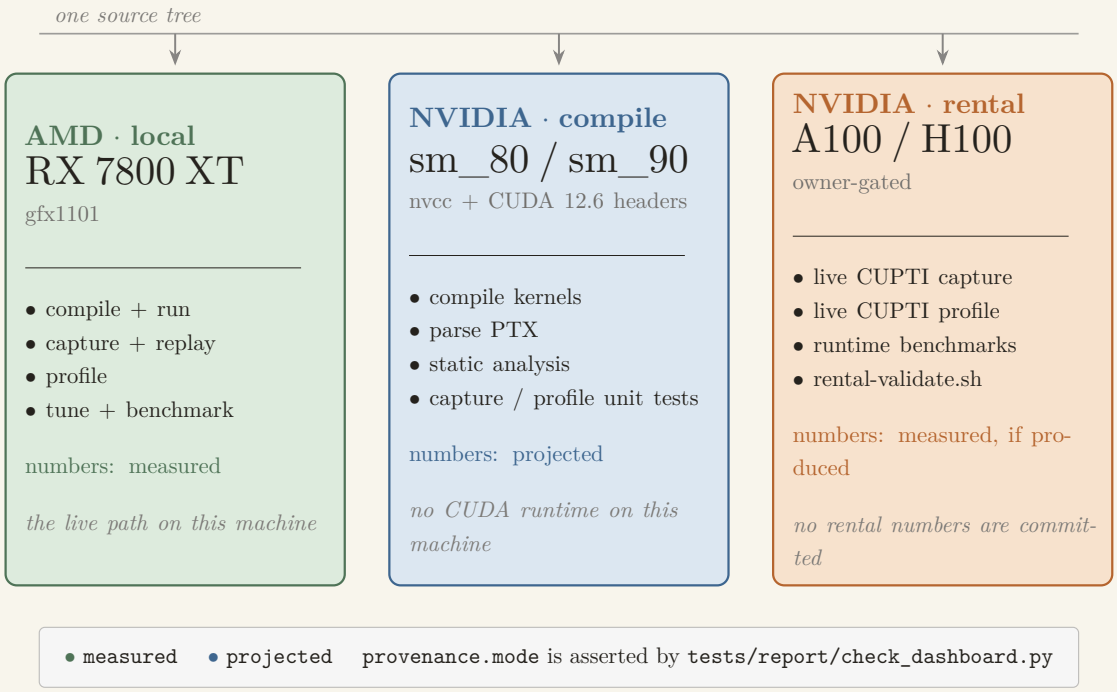


Figure 4. Three execution venues. The AMD column is live and measured, the NVIDIA compile column is projected, and the rental column is owner-gated and currently has no committed numbers.

3.3 The rental gate

Live NVIDIA capture and profiling run only on a rented A100/H100-class host through `scripts/rental-validate.sh`. The script is never called by `ci.sh` and refuses to run when `CI=true`. It prints the command sequence in `--dry-run` mode, then, in exec mode, checks the build tree, probes CUDA and CUPTI, runs the `gpu-nvidia-rental` tests, and writes a timestamped summary under `reports/rentals/`. Gating is procedural, not authenticated: the script checks the environment, not the identity of the operator. The repository held no rental numbers at consolidation time.

3.4 Capture mechanisms

The three capture mechanisms share the `libs/capture-core` session model and `BundleWriter`. This is what makes a bundle written by any mechanism parse identically.

3.5 The CUDA 13 migration surface

The suite pins CUDA 12.6 and `CUPTI_API_VERSION` 24. The CUDA 13 surface is documented so that a header bump is a compile error at one place, not a silent parse shift. The single place that reads raw CUPTI activity records is `tools/profile/include/kernscope/profile/record_adapter.h`.

- **Kernel records.** CUDA 12.x delivers `CUpti_ActivityKernel9`; 13.x delivers `CUpti_ActivityKernel10`. The 12.x struct moves to `cupti_activity_deprecated.h`. Kernel10 is Kernel9 plus a tail `uint8_t isDeviceLaunched` (and three padding bytes). The parser is a template over the delivered struct, selected by `CUPTI_API_VERSION`; offsets are pinned by `static_asserts` and `isDeviceLaunched` is read by SFINAE, so a 12.x build compiles against Kernel9 and treats the absent field as zero.

Mechanism	Vendor	Local status	Transport and scope
CUPTI callback/injection	NVIDIA	compile + unit only; rental-gated live	`CUDA_INJECTION64_PATH` or direct `libcupti.so`; Memory4 allocation records; `cuFuncGetParamInfo`; module-load callback.
LD_PRELOAD shim	NVIDIA	compile + unit; mock runtime locally	PLT interposition of `cuda*`/`cu*`; `cudaMalloc`/`cudaFree`; fatbin/`cuModuleLoadData`.
rocprofiler-sdk intercept	AMD	live on gfx1101	`LD_PRELOAD` plus intercept tables; HIP allocations; AMDGPU metadata; hsaco code objects; bitwise replay validated.

Table 4. Capture mechanisms and their local status. The NVIDIA paths never run live on this machine.

- **Removed APIs.** The CUPTI Event API and Metric API are removed in CUDA 13 and return `CUPTI_ERROR_LEGACY_PROFILER_NOT_SUPPORTED` (38). The PC-sampling ACTIVITY kind is replaced by `cupti_pcsampling.h`. nvprof and the NVIDIA Visual Profiler are gone. The build forbids the Event and Metric APIs; all profiling data comes from the Activity API.
- **Replacements.** The Profiling API and PerfWorks Metric API are replaced by the CUPTI Range Profiling API and the Profiler Host API. Multi-subscriber V2 APIs arrive in CUDA 13.3+; the 12.x single-subscriber path in `libs/cupti-sub` degrades to the fallback when a second subscriber is active.
- **Record-kind discipline.** Only `CUPTI_ACTIVITY_KIND_CONCURRENT_KERNEL` (10) is enabled. The serializing `KERNEL` kind (3) changes collection behavior and is never used.
- **Version codes.** `CUPTI_API_VERSION` 24 corresponds to CUDA 12.6; CUDA 13.0 reports 130001.

3.6 What the hardware split implies

The local machine can measure HIP behavior, PTX text analysis, static IR verification, and compile-only CUDA artifacts. It cannot measure NVIDIA runtime behavior. Every performance number in this report is therefore one of: a live gfx1101 measurement, a static projection from IR or PTX, or a correctness result from a compile gate. Section 13 keeps those categories separate.

4 The shared libraries

4.1 Architecture database and record store

`libs/arch` is an interface library of per-architecture constants: SM counts, register file sizes, shared-memory budgets, warp widths, and the measured gfx1101 peaks. It is the only place that answers "how big is the register file on this part." `libs/db` is the JSONL record store. Records are keyed by `gpu_arch`, `kernel_family`, and a shape hash, and the store supports `query_best` over the recorded timings. The tune cache and the benchmark/replay records both use it.

4.2 PTX and AMDGPU parsers

`libs/ptxparse` is a hand-written PTX lexer and parser: modules, entry functions, register declarations, instructions, and operand forms. It is used by the analyzer, the pass delta, and the auxiliary PTX dump source. `libs/amdparse` uses LLVM 21's MC layer to parse AMDGPU assembly and recover instruction, register, and operand facts. Both parsers preserve enough source text for the analyzer to emit evidence paths and finding identifiers.

4.3 Occupancy and roofline

`libs/occupancy` implements the min-wins occupancy model. For a block size and a kernel's register and shared-memory demand it computes resident blocks per SM from the thread, warp, shared-memory, register, and hard block limits, then reports the binding limiter. The NVIDIA register limit uses the architecture's allocation granularity:

$$\text{blocks}_{\text{regs}} = \left\lfloor \frac{65536}{\text{granule}(\text{regs per thread}) \times \text{threads per block}} \right\rfloor .$$

The AMD path models waves per SIMD and SIMD count, with the RDNA 3 VGPR file resolution described in the project notes. Occupancy is explicitly theoretical: it does not

predict achieved occupancy under memory stalls.

`libs/roofline` computes arithmetic intensity, the ridge point, the compute and memory ceilings, the bound class, and a position in $(0, 1]$:

$$\text{position} = \min\left(1, \frac{\text{AI}}{\text{ridge}}\right), \qquad \text{ridge} = \frac{\text{peak FLOP/s}}{\text{peak bytes/s}}.$$

The engine refuses to produce a position when the architecture has no measured peaks; `roofline.ok` is false rather than a datasheet guess.

4.4 Benchmark harness and runtime shim

`libs/bench` implements time-based warmup and repetition budgets, an L2 flush between trials, adaptive trial counts, and median plus p20/p80 quantiles. It is the timing method for the tuner, capture replay benchmark, and peak microbenchmarks. `libs/gpuruntime` is a thin CUDA/HIP runtime wrapper. It exposes the allocation, copy, launch, and synchronization surface the capture and replay code needs, with compile-only CUDA and mock backends so the shim can be exercised without an NVIDIA device.

4.5 Report engine and provenance

`libs/report` writes the self-contained JSON and HTML report form used by the tools. It sets `schema_version` to `"1.0"` and provides the per-module provenance object with a `mode` and a `measured` boolean. The report is self-contained: inline CSS and JavaScript, no external assets, and an embedded JSON payload under a stable script identifier, so a checker can extract and validate it without network access.

4.6 Capture backends

`libs/capture-core` owns the session model, the event log, the bundle writer and reader, the VA-faithful snapshot policy, and the patch table. It also builds the NVIDIA LD_PRELOAD shim and a mock CUDA runtime used by the unit tests. `libs/capture-hip` is the AMD shared object: it registers rocprofiler-sdk intercept tables, records dispatches and code objects, and writes the same bundle model. `libs/cupti-sub` is the shared CUPTI subscription layer used by the NVIDIA capture path and the profiler; it enforces a single-tool scope contract because the 12.x CUPTI subscription is single-subscriber.

4.7 Dependency rules

Nothing in `libs/` depends on a tool. The tools link libraries, not each other. The only cross-tool relationships are runtime file contracts: a bundle written by capture, a report written by a tool, a JSONL record written by the tuner. This is why the loop can be exercised by separate executables and why a tool can be replaced without rebuilding the stack.

Part II

The tools

5 C1: static analyzer

`kernscope-analyze` consumes PTX or AMDGPU assembly text and emits one kernel report plus a suggestion list. It is a static tool: it estimates occupancy and memory behavior from the instruction stream and the architecture database, and it labels every estimate as a metric rather than a measurement.

5.1 Inputs, CLI, and precedence

```
kernscope-analyze <file.ptx|file.s|file.isa> [options]
  --arch <sm_XX|gfxXXXX>   required for AMDGPU; PTX defaults to .target
  --block <N>              launch block size for occupancy
  --json | --text | --html output format (default --text)
  --source <file.cu|file.hip>  source stem for the kernel label
  --version | --help
```

The analyzer recognizes `sm_75`, `sm_80`, `sm_86`, `sm_89`, `sm_90`, `sm_100`, `sm_120`, `gfx90a`, `gfx942`, and `gfx1101`. A `.ptx` file goes through the PTX parser; `.s` and `.isa` files go through the AMDGPU parser; other extensions are content-sniffed. A directory input may carry `manifest.json` with `arch`, `block_size`, and a `ptxas_v_report`; command-line flags override the manifest. Without `--block`, occupancy emits the `block_size_unknown` diagnostic rather than guessing.

Register pressure prefers `ptxas -v` when a report is supplied. Otherwise the analyzer uses the declared `.reg` counts plus the `.local` frame, an upper bound: ptxas can reuse registers, so the proxy can overestimate. The AMDGPU path uses `scratch_load` and `scratch_store` for spills and does not have a `next_free_vgpr` value in plain assembly, so it does not invent one.

5.2 The ten analyses

The occupancy estimate is:

$$\text{blocks} = \min\left(\left\lfloor \frac{\text{threads}_{\max}}{\text{threads per block}} \right\rfloor, \left\lfloor \frac{\text{warps}_{\max}}{\text{warps per block}} \right\rfloor, \left\lfloor \frac{\text{smem}_{\max}}{\text{smem per block}} \right\rfloor, \text{blocks}_{\text{regs}}, \text{blocks}_{\text{hard}}\right),$$

and the binding term is reported as the limiter. The bank-conflict metric uses 32 banks of 4 bytes: the bank index is (word $\div$ 4) mod 32, and the conflict way is lanes divided by distinct banks. Coalescing uses the 128-byte-sector model and reports a byte-efficiency ratio; it does not claim a hardware counter.

5.3 Suggestion rules

The suggestion engine owns `report.suggestions`. It clears the list and emits at most one suggestion per triggering finding, in a fixed priority order, with an evidence text and a `triggered_by` link.

5.4 Report and golden result

The report is a JSON object with `kernel`, `metrics`, `limiter`, `warnings`, and `suggestions`; the schema is summarized in Appendix D. The committed golden fixture is a fused-attention-style CUDA kernel whose PTX targets sm_86: occupancy 9 blocks $\times$ 2 warps/48 = 0.375, limiter `registers`, 102 registers per thread, coalescing efficiency 0.5, a 68-word shared stride that maps to banks $\{0, 4, \ldots, 28\}$ with way 4, two `bar.sync` instructions, 1024 FP32 FMAs, zero spills, and three warnings plus three

Analysis	What it computes	Finding
Register pressure	Per-thread registers, warp allocation, local frame.	`spill`, `cannot_estimate_registers`
Occupancy + limiter	Blocks per SM from thread, warp, smem, register, and hard limits.	`occupancy`, `limiter`
Global coalescing	128 B sectors, used versus transferred bytes per warp.	`coalescing`, `coalescing_unknown`
Shared banks	Bank index, distinct banks, conflict way.	`bank_conflict`, `bank_conflict_unknown`
Warp divergence	Branch target and post-dominator analysis over thread-dependent predicates.	`possibly_divergent_branch`, `divergent_inner_loop`
Barriers	`bar.sync`, `bar.warp`, cluster barriers, density per loop, reachability under divergence.	`divergent_barrier`
Instruction mix	FP32/FP16/INT/LSU/SMEM/tensor/control histogram.	`mix_pct_*`
Tensor cores	PTX `mma`/`wmma`/`wgmma`/`tcgen05`; AMD `v_mfma_*`/`v_wmma_*`.	`no_tensor_core_usage`, `tensor_ops`
Spills	`ld.local`/`st.local`, local frame bytes, AMD scratch ops.	`spill`, `spill_bytes`
Latency	Static critical-path estimate from microbenchmark-derived op latencies.	`latency_bottleneck`, `latency_unknown_arch`

Table 5. The analyzer's ten analyses. The occupancy model is the min-wins formula from the occupancy library; the latency estimate is explicitly approximate.

Rule	Trigger and recommendation	Evidence
`tile_k_32`	Bank conflict of way ≥ 2; set the shared tile to 32 words.	Current stride, banks, shared bytes
`split_accumulator`	Register limiter and at least 16 FP32 FMAs; split the accumulator.	Registers, FMA count, fragment size
`hoist_predicate`	Divergent branch inside an inner loop; hoist the predicate.	Branch site, predicate dependence
`vectorize_coalescing`	Coalescing efficiency below 1.0 and no bank conflict; target 128 B per warp.	Efficiency, sectors, vector width

Table 6. Suggestion rules. The priority order is the table order. The engine drops instruction-mix informational findings so a suggestion is tied to an actionable warning.

suggestions. The sm_86 fixture is a parser corpus artifact; it is independent of the local `nvcc` capability pin.

6 C2: correctness-gated autotuner

`kernscope-tune` searches a kernel family's configuration grid for the fastest variant that passes a bf16 golden gate. It reports the winner with roofline and occupancy evidence, the full variant list, and a reproducibility bundle. It is not a blind search: infeasible configurations are rejected before compilation, and wrong kernels are never benchmarked.

6.1 Families and search spaces

Family	Auto grid
`matmul`	$(BM, BN) \in \{(64,64),(128,128)\} \times BK \in \{16,32\} \times warps \in \{1,2,4\} \times stages \in \{1,2\}$: 24 configs. The full grid expands to 4608 combinations over BK 8–64, stages 1–4, vector width 1–4, XOR swizzle, and unroll 1–4.
`bias_gelu`, `softmax`, `layernorm`, `reduction`	block $\in \{128,256,512,1024\}$ $\times$ vector width $\in \{1,2,4,8\}$; softmax and layernorm add a one-pass/two-pass mode, reduction adds strided/tree.
`flash_attention`	$Br \times Bc \in \{32,64\}^2$ $\times$ stages $\{1,2\}$ $\times$ vector width $\{1,2,4\}$ $\times$ shared-memory layout (padding/XOR) $\times$ causal on/off.

Table 7. Tuner families and their search spaces.

Each family owns a template generator that validates membership in the search space and checks the `libs/arch` budgets (registers, shared memory, warp/block divisibility, vector tiling) before compilation. A rejection is a reason string, not a compiler error.

6.2 The pipeline

A candidate moves through five stages:

1. **Cache hit.** A config already in the tunecache is replayed without compiling.

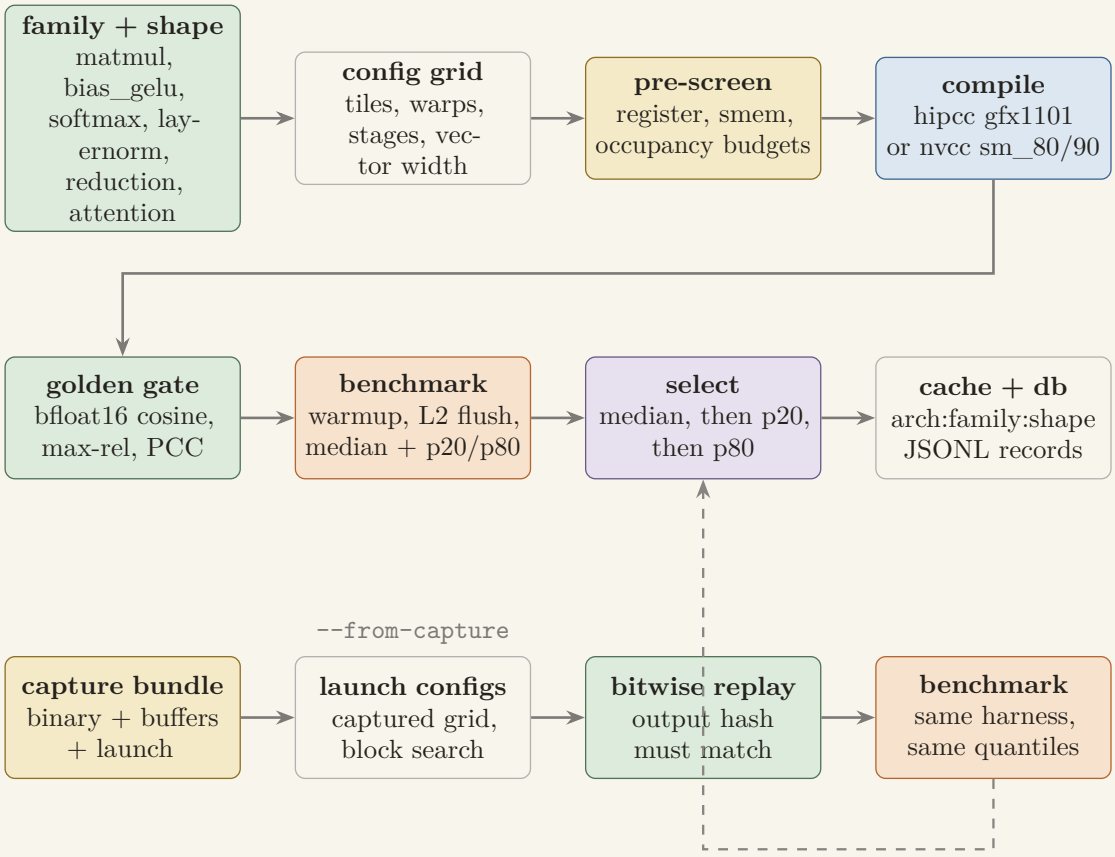


Figure 5. The tuner pipeline. The template path searches source variants; the capture path searches launch configurations over a captured binary. Both use the same benchmark and selection rules.

2. **GPU-free pre-screen.** The generator validates the config, then `libs/occupancy` rejects zero-resident-block configs and configs below `--min-occupancy`.
3. **Compile.** The family generator emits a single-source `.hip` file; `hipcc --genco` compiles it for gfxXXXX, or `nvcc -c` for sm_XX.
4. **Golden gate.** The kernel runs on the live HIP device through the template runner and the bf16 reference. The gate passes only when cosine ≥ 0.999, maximum relative error $\leq 10^{-2}$, and PCC ≥ 0.99. A failure discards the variant; it is never timed.
5. **Benchmark and select.** `libs/bench` warms up, flushes L2, adapts the trial count, and reports median and p20/p80. The best config wins by median, with p20, p80, and enumeration order as tie-breakers.

The gate shape is the family's golden shape (64^3 for matmul), deliberately decoupled from the benchmark shape. The cache is keyed `gpu_arch:family:shape_hash` and stamped with the minimum-occupancy floor that produced it, so a floor-1.0 entry never replays for a floor-0.0 run. `--no-cache` skips the read but still writes the new entries.

6.3 Outputs and reproducibility

`--json` emits the best config and timings, the baseline (the first enumerated measured config), the speedup over that baseline, every measured variant sorted by median, the pipeline counts, the roofline position and bound, and the winning occupancy and limiter. `--text` is the owner-facing one-screen form. `--html` is a self-contained variant comparison with the model embedded under `report-data` and the evidence under `tune-evidence`.

`--emit-repro` writes a standalone directory with `run.sh`, `README.md`, and the winning config. The script regenerates the source, repeats the exact compile command, and re-runs the golden gate at the winning geometry. The nixpkgs revision from `devenv.lock` is embedded and checked against `KERNSCOPE_EXPECT_NIXPKGS_REV` as a warning. The repro path needs a live HIP device and exists for matmul plus the four elementwise families; `flash_attention` has no generator CLI and is rejected with a clear message.

6.4 Capture-tune mode

A capture bundle contains a binary, arguments, and buffers, but no source. Template tuning cannot consume it; the launch configuration is the only searchable degree of freedom. `kernscope-tune --from-capture` restores the captured launch, enumerates block sizes at the captured grid, pre-screens the limits (threads per block ≤ 1024, grid x $\leq 2^{31}-1$, grid y/z ≤ 65535, shared memory within the architecture budget), replays each survivor bitwise, and times only the survivors. The captured baseline is candidate 1, so `best.speedup` is always at least 1.0. The record key is `(arch, sha256(kernel name), sha256(bundle identity))`; the binary and buffers stay byte-identical to the capture.

6.5 Measured evidence

In the demo, capture-tune explored four candidates over the captured layernorm bundle. All four validated bitwise; the best block was 256 with a median of 0.01206 ms against a 0.01208 ms baseline, a speedup of 1.0017×. The verifier-gated template run on bias_gelu at 4096×4096 enumerated 13 variants, compiled 12, and gated one before compilation. The best config (block 256, vector width 4) ran at 0.17184 ms against a 0.20332 ms baseline, a 1.1832× speedup, occupancy 1 (limiter registers), arithmetic intensity 2.333, and roofline position 0.0845 in the memory-bound class.

The dashboard records a separate measured matmul search with 15 variants, a best

median of 415.324 μs, and a worst median of 3.106 ms. The per-run JSONL for that search is not committed, so the report quotes the dashboard value and labels it as such rather than reconstructing a raw record.

7 C3: static verifier

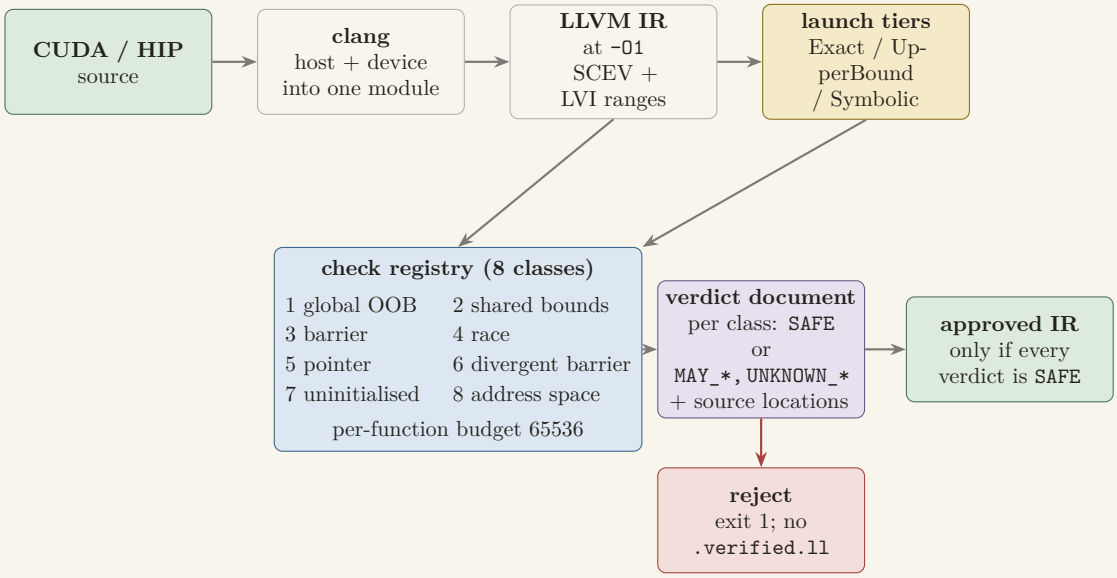
`kernscope-verify` compiles CUDA or HIP source to LLVM IR at `-O1` and applies eight may-analyses. It does not prove a kernel correct; it proves specific bounds and ordering facts when it can, and it refuses approval when it cannot.

7.1 Ingestion and launch tiers

By default the host and device halves are compiled separately and linked into one module, so the launch site is visible. The host pass supplies the launch configuration through the `__cudaPushCallConfiguration` or `cudaLaunchKernel` call. HIP uses clang 21 with `--hip-path`. `--device-only` is a degraded mode: launch dimensions are lost unless the kernel carries `__launch_bounds__`.

Launch dimensions resolve into three tiers:

1. **Exact**: constant dimensions at the launch site, giving exact ranges for `threadIdx` and `blockIdx`.
2. **UpperBound**: `__launch_bounds__` or `nvvm.maxntid` provides a sound upper bound on `blockDim`.
3. **Symbolic**: runtime dimensions produce a `SYMBOLIC_DIMS` verdict and a runtime-check suggestion, never a claim of safety.



The UX follows the Linux eBPF verifier: reject with a line-level reason, track types and ranges, bound the explored states, and degrade to a non-proof verdict instead of hanging or claiming safety.

Figure 6. The verifier pipeline. The launch tiers feed the diagnostic engine; the check registry produces per-class verdicts; approved IR is written only when no error-class verdict remains.

ID	Class	Plugin	Verdict vocabulary
1	Global out-of-bounds	`checks/oob`	`SAFE`, `MAY_OOB`, `SYMBOLIC_DIMS`
2	Shared-memory bounds	`checks/oob`	`SAFE`, `MAY_OOB`, `SYMBOLIC_DIMS`
3	Barrier correctness	`checks/barrier`	`SAFE`, `BARRIER_MISMATCH`
4	Race-prone shared writes	`checks/race`	`SAFE`, `MAY_RACE`, `UNKNOWN_RACE`
5	Illegal pointer arithmetic	`checks/ptr`	`SAFE`, `PTR_OOB`
6	Divergent barriers	`checks/barrier`	`SAFE`, `DIVERGENT_BARRIER`
7	Uninitialized reads	`checks/ptr`	`SAFE`, `MAY_UNINIT_READ`
8	Invalid address-space casts	`checks/ptr`	`SAFE`, `INVALID_ADDRSPACE_CAST`

Table 8. The eight check classes and their stable IDs. Engine-degraded verdicts use IDs at or above 100: symbolic dims (100), SCEV reconstruction failed (101), and budget exceeded (102).

7.2 The eight check classes

The design follows the Linux eBPF verifier's user experience (Linux kernel, 2026): reject with a concrete line-level reason, track types and ranges, cap the explored states, and degrade to a non-proof verdict rather than hanging. Every non-`SAFE` verdict carries diagnostics with a source location, the enclosing kernel, the offending instruction, and an evidence path where one exists. The per-function explored-state budget defaults to 65536 and is settable with `--budget`.

7.3 Verdict semantics and approved IR

`SAFE` is the only approval-class verdict. Every other verdict, including the engine-degraded ids, means no bound was proven. The CLI exits 1 and writes no approved IR. If the error-class count is zero, the verifier projects the verified module onto its device code: it keeps the kernels and reachable device helpers, drops host functions, and rewrites the global constructor and used lists. The result is `<base>.verified.ll`. The projection matters for the handoff because the proofs apply to device code and the AMDGPU backend rejects host `addrspace(0)` allocas.

7.4 Soundness scope

All eight checks are may-analyses at `-O1`. In scope: global and shared accesses whose indices are bounded by SCEV and the launch-dimension lattice; barrier counts and divergent conditional barriers; intra-kernel shared-memory access pairs partitioned by barrier phases; pointer arithmetic, uninitialized reads, and address-space casts. Out of scope: data-dependent indices (reported `MAY_OOB` or `UNKNOWN_RACE`, never `SAFE`), runtime-parameter allocations (an index-bounded `SAFE` carries an explicit note), atomics, and host-side allocation tracking.

The known false positives include identical thread-indexed expressions that a value-numbering pass could merge, disjoint arithmetic that SCEV cannot prove disjoint, unbounded thread indices that overlap everything, loop-carried phases, and an unknown-range thread access paired with a uniform address. The known false negatives include data-dependent indices (named `UNKNOWN_RACE`, not proven safe), uniform-address value races, lone data-dependent stores, divergent barriers still treated as separators, and `__syncwarp`, `__threadfence_block`, and atomics not modeled as synchronization. Inter-block races are out of scope. The future path is HGRD-style dominance-based barrier intervals, Faial-style behavioral types, and a GPUVerify-style two-thread reduction; none

is implemented here.

7.5 Corpus and evidence

The corpus has 19 source files and 34 consolidated manifest entries across the four check plugins. The lit gate runs the CLI `--json` over every corpus kernel and diffs the verdicts against the manifest read from disk; the CLI never reads the manifest, so the test is not self-reporting. The dashboard summarizes the projection as 18 `SAFE`, 4 `MAY_OOB`, 4 `PTR_OOB`, 2 `MAY_RACE`, 2 `MAY_UNINIT_READ`, 1 `DIVERGENT_BARRIER`, 1 `BARRIER_MISMATCH`, 1 `UNKNOWN_RACE`, and 1 `INVALID_ADDRSPACE_CAST`. In the demo, the verifier gate removed one risky bias_gelu variant before compilation while 12 others compiled and benchmarked.

8 C4: capture and replay

`kernscope-capture` records what a kernel launch observably did and lets a later process replay it. The recorded bundle is the input to the analyzer, the capture-tune search, and a standalone reproducer. The central constraint is VA fidelity: the replay must reconstruct the original pointer graph, not just the bytes.

8.1 Mechanisms

Table 4 lists the three mechanisms. All three share the `libs/capture-core` session model and bundle writer, so the reader does not need to know which one produced a bundle. Vendor selection is `--vendor`, then `$KERNSCOPE_VENDOR`, then the first capture shared object next to the CLI (HIP preferred), then an error.

8.2 Bundle format v1

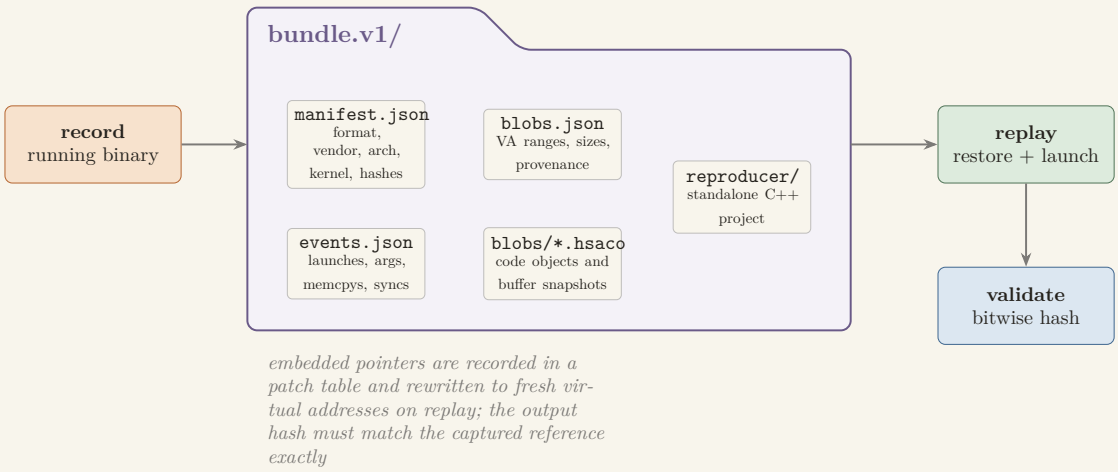


Figure 7. Bundle v1. The manifest names the vendor and mechanism; the event log records API calls, allocations, launches, copies, memsets, streams, modules, and parameter metadata; the blob index carries a SHA-256 and a patch table for every snapshot.

A v1 bundle is a directory:

```
<name>.kcap/
  manifest.json
  events.json
  blobs.json
  blobs/b0001.bin
```

`manifest.json` must have `format_id="kernscope-capture"`, `schema_version="1.0"`, `vendor` in {nvidia, amd, mock}, `capture_mechanism` in {cupti, preload, hip-sdk}, `snapshot_policy="va-faithful"`, and a `size_governance` object with a default maximum snapshot of 1 GiB. A total over the cap stops further blob writes, sets `aborted=true`, records a reason, and leaves a loadable partial bundle.

`events.json` is an ordered log. The record types are API counts, allocation and free, unknown free, launch, copy, memset, stream, fatbin, cubin, and parameter info. A launch carries the kernel name, the launch mechanism, grid/block/shared memory, stream and graph handles, and a per-parameter argument table. Each argument is a pointer or scalar, with a size, a byte-exact host copy, and the original virtual address for pointer slots.

`blobs.json` indexes the snapshots. Each blob has an id, file, role, VA, size, SHA-256, and patch table. Roles are kernel binary, input, output, argument, and state. The reader recomputes every hash on read and rejects a mismatch. The patch table is the mechanism that makes replay portable: every 8-byte little-endian value in a blob that equals a known allocation VA is recorded as a patch, and replay rewrites only the entries present in the allocation remap. A `__device__` global that happens to contain another global's address is out of scope.

8.3 Replay, validation, and benchmark

The replay model resolves buffers from allocations, H2D inputs, D2H markers, and v1 snapshots, then restores the binary, the argument plan, and the launch geometry. HIP/HSA assigns virtual addresses, and the current runtime wrapper does not expose CUDA VMM placement, so replay remaps on both vendors. It allocates fresh buffers, computes old-to-new VA remapping, applies the patch tables before upload, rewrites pointer argument slots through the same table, and launches the captured grid, block, and shared memory.

Validation is bitwise by default: byte compare plus SHA-256, with a hash diff on failure. Tolerance mode interprets outputs as bf16 chunks upcast to fp32 and passes only when cosine ≥ 0.999, maximum relative error $\leq 10^{-2}$, and PCC ≥ 0.99. References come from v1 output snapshots or from `--ref` in output-buffer order; a missing reference is an error, not a pass.

`bench` performs one validation launch before timing and refuses to time an invalid replay unless `--no-validate` is given. It uses ten repetitions by default, an L2 flush, event timing, and median plus p20/p80. Each run writes one `libs/db` record keyed by architecture and kernel hash, with `provenance.mode="measured"`.

8.4 Reproducer and size governance

`emit-repro` writes a standalone plain-CMake C++ project with the kernel binary, inputs, a `main.hip` or `main.cu`, a `CMakeLists.txt` pinned to the manifest vendor and architecture, and a README. It re-points pointer arguments at fresh buffers and launches the captured geometry. Size-unknown buffers become deterministic synthetic inputs, marked so a reader does not mistake them for bit-exact data. The NVIDIA reproducer is compile-only.

8.5 CUDA/HIP field mapping

The mapping contract compares what the two vendors can both express, after normalization. Kernel names are compared after stripping the `.kd` suffix and demangling. Grid,

block, and shared memory compare directly. The default stream normalizes to 0; non-default streams get a 1-based id. Pointers compare as 1-based allocation ordinals, not raw VAs. Scalars compare by size and byte-identical value; input blobs compare by SHA-256 after H2D call order. Allocations compare as a sorted multiset of sizes; copies compare by kind (H2D, D2H, D2D). Binary blobs are compared per vendor only; a cubin is never compared with an hsaco. API counts, raw pointers before normalization, and timestamps are not compared.

8.6 Measured evidence

The demo captured the hot layernorm kernel `_Z15layernorm_fusedPKtS0_S0_Ptfi.kd` into a v0 text bundle. The v1 layout is the current reader contract and the capture-tune path accepts it, but the record CLI in this tree still writes the v0 form; the gap is flagged in Section 15.1. The launch was at grid $1024 \times 1 \times 1$ and block $128 \times 1 \times 1$. The bundle has three blobs. Replay reproduced the output bitwise; the reference SHA-256 is `a0027ed9a8ef2e3453aeb194aa64108c1f300a87fbfa292cca5222a725449eec`. The capture-tune stage then explored four block sizes over the same bundle, validated all four bitwise, and selected block 256. This is the strongest cross-tool evidence in the report: the analyzer's stage-3 findings and the tuner's stage-4 candidates both refer to the same captured code object and launch.

9 C5: LLVM passes

`kernscope-pass` is an out-of-tree LLVM 21 pass plugin. It is loaded by both driver paths, `opt -load-pass-plugin` and `clang -fpass-plugin`, and it uses the New Pass Manager's `PassPluginLibraryInfo` entry point. The `kernscope-pass-report` CLI runs a fixed showcase pipeline and measures the before/after delta with the analyzer.

9.1 Plugin structure

A pass module subclasses `llvm::PassInfoMixin` and registers a callback with `REGISTER_PASS_CALLBACK` at file scope. The callback receives the `PassBuilder` of the loading process and wires the module into the pipeline. The registry is a process-global singleton; the plugin entry point does not need to know the module names in advance. The lit suite exercises both `opt` and `clang` loading so an ABI change fails at the loader rather than at the first transformed kernel.

9.2 The passes

Pass	Transformation or analysis
`kernscope-promote`	Global-to-shared promotion for tiles reused in a loop.
`kernscope-tile`	Loop tiling over constant-trip, unit-stride `\{s,+,1\}` windows.
`kernscope-vectorize`	Rewrites scalar load/store chains into vector operations when alignment and contiguity permit.
`kernscope-pad`	Shared-memory padding to break a known bank-conflict stride.
`kernscope-rle`	Redundant-load elimination and address simplification.
`kernscope-coalesce`	Analysis of per-access coalescing class; no rewrite.

Table 9. The pass modules. The coalescing analysis is shared with the analyzer's 128-byte-sector model so before/after numbers use the same definition.

The default showcase pipeline is `kernscope-tile`,`kernscope-promote`,`kernscope-vectorize`,`kernscope-pad`,`kernscope-coalesce`. Not every pass fires on every kernel; a no-transform result is a valid outcome and is recorded as such. The tile pass, for example, declines a layernorm loop whose induction variable is not a canonical `\{0,+,1\}` phi.

9.3 The delta report

`kernscope-pass-report --report-pipeline <gemm|layernorm|softmax>` runs clang to before-IR, the plugin pipeline to after-IR, and `llc` to before/after PTX. It then invokes `kernscope-analyze` on both PTX files and emits a delta report with register estimate, shared-memory bytes, coalescing class, bank-conflict way, vector width, and vectorized global loads. Every metric is labeled improved, unchanged, or not applicable. The report's provenance is `projected`: the IR and PTX are compile-only artifacts, and the note says the measured gfx1101 analog lives in the `libs/db` records.

On layernorm, vector width goes from 1 to 4, vectorized global loads from 0 to 12, uncoalesced accesses from 64 to 16, and the register estimate stays at 175. The pass report records 12 vector chains and 48 vectorized loads. gemm and softmax are correctly left no-transform, and all three kernels compile to PTX with exit 0 after the pipeline. The tile pass's refusal is recorded with its reason rather than hidden.

10 C6: profiler

`kernscope-profile` answers three questions from a running binary: which kernels ran, how long each took, and what the compiler-visible facts suggest about the bottleneck. It does not require hardware counters. It traces, it enriches, it ranks, and it diagnoses from the activity records and the architecture database.

10.1 Pipeline

1. **Trace.** The target runs under `LD_PRELOAD` of the rocprofiler-sdk tracer; each kernel dispatch and HIP API call becomes a JSONL record. On NVIDIA the equivalent path is the CUPTI activity tracer, which is compile- and unit-tested locally and rental-gated for live use.
2. **Enrich.** Each kernel record receives occupancy and limiter estimates from `libs/occupancy`, code-object VGPR/SGPR/LDS facts where available, and explicit degradation markers where not.
3. **Rank.** Kernels are grouped by name and ranked by total time (then median, then name); the top ten are kept.
4. **Diagnose.** The rule engine classifies memory, latency, and compute behavior for known families, reports the occupancy limiter, and degrades to `insufficient_evidence` rather than inventing a cause.
5. **Report.** `--json` emits the model, `--text` the owner-facing table, and `--html` a self-contained page with the embedded JSON.

10.2 Diagnostics

A `memory_bound` hint is emitted when the workload's arithmetic intensity is below the measured ridge point. A `latency_bound` or `compute_bound` hint follows the same roofline position and the instruction mix. An occupancy limiter hint is emitted when occupancy is below 1.0; the limiter is the binding resource. Sub-hints distinguish register bloat, shared-memory pressure, and a block too small to fill the machine. Every hint carries

its evidence: arithmetic intensity, bytes moved, ridge point, occupancy, registers, or the analyzer finding id when the analyzer was merged into the pipeline.

10.3 Measured evidence

Tracing the layernorm app produced three kernels and 19.6 μs of wall time. The hot kernel ran for 9.2 μs with 24 registers per thread, occupancy 1 (limiter registers), 1024 bytes of shared memory, grid $1024 \times 1 \times 1$, and block $128 \times 1 \times 1$. The profiler emitted a memory-bound hint with arithmetic intensity 1.375, a bytes-moved estimate of 1048576, and a ridge point of 27.614. The hint's `analyzer_cross_refs` in the aggregated report point at the stage-3 analyzer findings, including `possibly_divergent_branch`.

11 Bench, dashboard, and the evidence layer

11.1 Benchmark methodology

`libs/bench` is the one timing method in the project. It warms up the kernel, flushes L2 between trials, and adapts the trial count to a time budget while keeping a minimum sample count. It reports the median and the p20/p80 quantiles, not a mean, so a single slow launch does not move the result. The same harness times the peak microbenchmarks, the tuner variants, and the capture-replay benchmark. A correctness gate runs before timing in the tuner and in capture replay; the peak microbenchmarks are their own reference.

11.2 Measured machine peaks

The peaks in Table 3 drive the roofline engine. The fp32 figure is the register-resident FFMA loop; the DRAM read figure uses a 512 MiB array; the LDS figure is a read-plus-write loop. The device-to-device figure is reported as a range over the accepted runs. Clocks were recorded with `rocm-smi` but not locked, and the machine was not isolated from development activity. The numbers are therefore a measured description of this machine on 2026-08-14, not a datasheet claim.

11.3 Dashboard sections

The dashboard aggregates six sections. Each section has a title, an availability flag, a status, a provenance object, a summary, and a list of evidence paths.

Section	Mode	Summary at consolidation
`analyzer`	projected	Golden occupancy 0.375, 102 registers, limiter registers, 3 warnings, 3 suggestions.
`tune`	measured	15 matmul variants; best 415.324 μs, worst 3.106 ms.
`verify`	projected	34 entries, 19 files, 8 classes; 18 SAFE.
`capture`	measured	Grid $1024 \times 1 \times 1$, block $128 \times 1 \times 1$, 3 blobs, bitwise PASS.
`profile`	measured	3 kernels, 19.6 μs wall, hot 9.2 μs, memory-bound hint.
`pass`	projected	Layernorm vector width 1→4, uncoalesced 64→16; gemm/softmax no-transform.

Table 10. Dashboard sections and their provenance modes. The mode is a convention recorded by the aggregator; the report schema does not require it.

11.4 What the provenance does and does not enforce

The report schema (Section D) requires `kernel`, `metrics`, `limiter`, `warnings`, and `suggestions`. It does not require `schema_version` or `provenance`; those are report-engine and dashboard conventions. The per-section provenance object has a `mode` in {measured, projected, no_data} and a boolean `measured`. The dashboard validator asserts the six sections, the boolean, the evidence files, and the rendered module markers. The validator is not called by `ci.sh`; it is a separate check. The report states this because a provenance convention that is described as enforced when it is only intended is exactly the kind of claim this project is trying to avoid.

Part III

Evidence and operation

12 Testing, CI, and reproducibility

12.1 The four test layers

The gate is layered so a regression stops at the cheapest test that can see it:

1. **gtest / ctest.** C++ unit and integration tests carry one hardware label: `no-gpu`, `gpu-amd`, or `gpu-nvidia-rental`. The first two are the local gate; the third is owner-executed on a rental.
2. **lit / FileCheck.** The `check-kernscope` target runs the parser, pass, verifier-manifest, and CLI tests against committed IR and ISA corpora.
3. **NumPy bf16 golden harness.** `pytest tests/golden` compares generated kernels against reference outputs under the bf16 tolerance contract. It is the independent implementation of the C++ golden gate.
4. **Generated-code compile checks.** `scripts/compile_checks.sh` compiles the template variants with `nvcc` for sm_80/sm_90 (compile-only) and `hipcc` for gfx1101, then runs the live HIP variant through the golden gate.

12.2 The CI gate

`scripts/ci.sh` runs the stages in order and exits non-zero on the first failure. A stage whose artifact is absent prints `[SKIP]` and stays green, which is a bootstrap convenience, not a claim that the stage ran.

Stage	Check
a	`scripts/configure.sh` and `cmake --build build`.
b	`ctest -L "no-gpu|gpu-amd"` with hardware labels.
b2	Non-decreasing count guard against `scripts/ci-baseline.txt` (550).
c	`cmake --build build --target check-kernscope` (lit/FileCheck).
d	`pytest tests/golden -q` (NumPy bf16 goldens).
e	`scripts/compile_checks.sh` (nvcc sm_80/90, hipcc gfx1101, live HIP golden).
f	Committed fixtures total at most 64 MB (`du -sm tests/fixtures`).
g	NVIDIA-header guards: no vendored CUDA/CUPTI/NVTX header filenames or vendor copyright markers outside the documented exceptions.

Table 11. The CI stages. The count guard is non-decreasing, not exact: adding tests is always green, and removing a registration below the floor fails.

The `CI_SKIP_CONFIGURE=1` escape hatch skips configure and build. It exists for a bootstrap situation where the nix-store LLVM resolution is being repaired; it is owner-set and it makes the skip visible. The gate does not run the dashboard validator, and it does not run a secret scanner. Those are operational gaps, not hidden passes.

12.3 The master reproducibility script

`scripts/reproduce-all.sh` chains four stages: the lock check, `ci.sh`, `demo.sh`, and the dashboard. The lock check requires `devenv.lock` to exist and be tracked; it never regenerates the lock silently. The dashboard writes `reports/dashboard.html` and `reports/dashboard.json` with `SOURCE_DATE_EPOCH` defaulted to the HEAD commit time, so two runs on the same commit can be byte-identical.

A clean checkout is:

```
devenv shell
```

```
git clone https://github.com/meillaya/kernscope.git
cd kernscope
bash scripts/ci.sh
bash scripts/reproduce-all.sh
```

The first configure and build are the slowest (roughly ten minutes on the reference machine); later runs reuse `build/`. The committed fixtures, golden inputs, lit corpora, and schema files are the complete gate input. The earlier documentation used a local absolute clone path; this report states the public URL instead.

12.4 Owner-gated NVIDIA runs

Only `scripts/rental-validate.sh` runs NVIDIA code. It is never called by `ci.sh`. In `--dry-run` it prints the sequence: SSH to the rental host, check `nvcc`, detect the CUPTI version from `cupti_version.h`, run `ctest -L gpu-nvidia-rental`, and place evidence under `reports/rentals/`. In exec mode it refuses when `CI=true`, requires a configured build tree, probes `$CUDA_HOME`, `/usr/local/cuda`, and `/opt/cuda` for CUPTI, and writes a timestamped summary with the verdict, exit code, CUDA major, and CUPTI version.

The gate is procedural. The script checks the environment, not the operator's identity, and it assumes SSH keys are already configured. The repository ignores `reports/rentals/*` except the placeholder, so an owner who wants to commit a rental summary must force-add it. No rental numbers are claimed in this report.

12.5 Secrets and sensitive data

The repository ignores `private_key.pem`, `public_key.pem`, `.env`, and `.env.*` (with an exception for `.env.example`). The rental script does not manage keys. There is no secret scan in `ci.sh`; the rule is enforced by the ignore list and by review, not by an automated gate. Capture bundles are local-only and gitignored; snapshot sizes are capped at 1 GiB by default, and the fixture budget is 64 MB.

13 Evaluation

13.1 The demo run

`scripts/demo.sh` ran the loop on the local gfx1101 in 153 s against a 900 s budget. The aggregated report is the primary cross-tool evidence.

Stage	Status	Evidence
1 profile	ok	3 kernels, 19.6 μs wall, hot layernorm 9.2 μs, 24 registers, occupancy 1, `memory_bound` hint (AI 1.375, ridge 27.614).
2 capture	ok	Bundle `bundle.v0`, 3 blobs, grid $1024 \times 1 \times 1$, block $128 \times 1 \times 1$, bitwise replay PASS, reference SHA-256 `a0027e...49eec`.
3 analyze	ok	Captured AMDGPU ISA, 216 ALU ops, `possibly_divergent_branch` warning, three informational findings.
4 capture-tune	ok	4 candidates validated, baseline block 128 at 0.01208 ms, best block 256 at 0.01206 ms, speedup 1.0017×.
5 verify + pass	ok	13 bias_gelu variants, 12 compiled, 1 gated; best 0.17184 ms vs 0.20332 ms, 1.1832×; pass vector width 1→4, loads 0→12, uncoalesced 64→16.

Table 12. The five demo stages. The report's cross-tool block links the stage-3 analyzer finding ids to the stage-1 profiler hint.

The cross-tool link is the part that matters. The profiler's `memory_bound` hint carries `analyzer_cross_refs` with the finding ids `possibly_divergent_branch`, `no_tensor_core_usage`, `occupancy_not_applicable`, and `suggestions_not_applicable`, and the stage-3 evidence path. The analyzer ran on the captured kernel's disassembled ISA, not on a hand-picked example. The verifier removed one variant before compilation. The pass delta used the analyzer's own metrics. This is the concrete meaning of "the loop closes" in this report.

13.2 Analyzer and verifier evidence

The analyzer golden fixture is described in Section 5.4. The verifier corpus has 34 entries across 19 source files, with 18 `SAFE` verdicts and 16 non-`SAFE` verdicts spread across the bounds, race, barrier, pointer, uninitialized-read, and address-space classes. Both results are `projected`: they come from static analysis, not from a GPU run. The verifier's anti-self-report gate diffs the CLI's JSON against the manifest read from disk, so a corrupted expectation fails the lit suite.

13.3 Tuner and capture evidence

The capture-tune and verify-tune results in Table 12 are measured on gfx1101. The capture path is bitwise: every explored config must reproduce the captured output hash before it is timed. The template path is correctness-gated: a variant that fails cosine, relative error, or PCC is discarded, not timed. The measured speedups are small for capture-tune (1.0017×) and modest for the bias_gelu template search (1.1832×); the report quotes them as measured, not as a general speedup claim.

The dashboard records a separate matmul search with 15 measured variants and a best median of 415.324 μs. The raw JSONL for that run is not committed, so this is a dashboard-sourced value, not a value reconstructed from a raw record. The distinction matters because the dashboard's tune section lists source paths that are absent from the tree.

13.4 Pass and profile evidence

The layernorm pass delta is projected: vector width 1→4, vectorized global loads 0→12, uncoalesced accesses 64→16, registers unchanged at 175. The before/after PTX compiles cleanly, and gemm and softmax are correctly no-transform. The report does not claim a wall-clock pass speedup; it claims the static delta that the pipeline measures.

The profiler evidence is measured: 3 kernels, 19.6 μs wall, a 9.2 μs hot kernel, 24 registers, occupancy 1, and a memory-bound hint with arithmetic intensity 1.375 and ridge point 27.614. The hint is a compiler-aware diagnosis from activity records and the roofline, not from hardware counters.

13.5 Threats to validity

- **One measured platform.** Every live number is gfx1101 on one RX 7800 XT. The same kernel on CDNA or NVIDIA may behave differently.
- **Unlocked clocks.** The peak run recorded clocks with `rocm-smi` but did not lock them, and the machine was shared with development activity. The peaks are a description of this machine at that time.
- **Projected deltas.** The pass numbers are static IR/PTX artifacts. They are not wall-clock speedups.

- **Verifier scope.** The verifier is a may-analysis, not a formal prover. Its false-positive and false-negative classes are stated in Section 7.4.
- **Dashboard gaps.** The tune section's source files are absent; the matmul number is quoted from the dashboard, and the validator is not part of `ci.sh`. The report says so rather than presenting the dashboard as a complete raw archive.
- **Documentation consolidation.** The raw Markdown notes and the committed JSON/HTML evidence were folded into this report and removed from the tree. The values remain; the raw files do not. Regeneration is possible with the committed scripts on a gfx1101 machine.

14 Context and prior art

kernscope is an engineering synthesis, not a new verification calculus. Its design choices follow a body of work that the project's research briefs survey; this section states the main lineages and what kernscope borrows or declines.

14.1 Performance models and static analysis

The roofline model (Williams et al., 2009) supplies the compute and memory ceilings and the ridge point used by the tuner and profiler. Occupancy follows the CUDA programming guide's resource model (NVIDIA, 2026a), with the low-occupancy/high-ILP trade-off documented by Volkov (Volkov, 2010). Nsight Compute's Speed of Light vocabulary (NVIDIA, 2026d) is the diagnostic register the profiler adopts.

The analyzer's static estimates sit against a line of microbenchmark-driven architecture studies (Volta, Turing, Hopper, and CDNA3) and against simulators such as GPGPU-Sim and Accel-Sim (Bakhoda et al., 2009; Khairy et al., 2020) and binary instrumentation such as NVBit (Vijaykumar and et al., 2019). Those tools measure or simulate; kernscope reads the ISA text and the architecture database, which is cheaper and less precise. The analyzer's contribution is the reuse of the same architecture constants and metrics by the tuner, pass report, and profiler.

14.2 Autotuning and tensor compilation

The autotuner follows the algorithm/schedule separation of Halide (Ragan-Kelley et al., 2013), the search-based scheduling of TVM and Ansor (Chen et al., 2018; Zheng et al., 2020), and the tile-level autotuning model of Triton (Tillet et al., 2019). CUTLASS (NVIDIA, 2026c) and the vendor libraries set the performance bar; Kernel Tuner (van Werkhoven, 2024) is the closest standalone autotuner prior art. FlashAttention (Dao et al., 2022) motivates the memory-bound attention family and the IO-aware tiling direction. kernscope does not generate a general tensor program; it generates family-specific templates, gates them on correctness, and records the search in a cache that can be replayed.

14.3 GPU verification

GPUVerify (Betts et al., 2012, 2015) is the methodological anchor: the two-thread reduction turns race and divergence checks into sequential program verification. PUG, SESA, GKLEE, and ESBMC-GPU (Li and Gopalakrishnan, 2010; Li et al., 2014, 2012; Pereira et al., 2017) explore symbolic execution and bounded model checking. Faial (Liew et al., 2024) contributes behavioral types over index expressions and the finding that most data-race-free kernels need a thread-configuration assumption. HiRace and HGRD

(Jacobson et al., 2024; HGRD authors, 2026) bring dynamic and MLIR-level analyses. LLOV (Afzal et al., 2020) demonstrates static race freedom on LLVM IR. The Linux eBPF verifier (Linux kernel, 2026) supplies the interaction model: typed ranges, bounded state exploration, and a concrete rejection reason. MemorySanitizer (Serebryany and Stepanov, 2015) and the SPIR-V validator (Khronos Group, 2026) inform the uninitialized-read and well-formedness checks.

kernscope-verify is deliberately smaller: eight may-analyses over LLVM IR at `-O1`, with the launch configuration ingested from the host half and a three-tier exact/upper-bound/symbolic contract. It does not claim the soundness of GPUVerify or Faial, and its false-positive and false-negative classes are published rather than hidden.

14.4 Capture and replay

Kerncap (Ramos and Lowery, 2026) is the closest complete system: HSA-level extraction and VA-faithful replay for HIP and Triton. Mneme (Parasyris et al., 2023) records and replays kernels for tuning; the LLNL Record-Remix-Replay line (LLNL, 2026) and Tally (Tally authors, 2024) explore the same design space. CUPTI (NVIDIA, 2026b) and rocprofiler-sdk (AMD, 2026) provide the vendor tracing surfaces. kernscope's contribution is the shared bundle model across CUPTI, LD_PRELOAD, and rocprofiler-sdk, the explicit patch table for embedded pointers, and the use of the same bundle by the analyzer, capture-tune, and the standalone reproducer.

14.5 LLVM passes and profiling

The pass plugin uses LLVM's New Pass Manager and the out-of-tree `PassPluginLibraryInfo` ABI (Lattner and Adve, 2004; Lattner et al., 2021). The transformations are deliberately narrow: promotion, tiling, vectorization, padding, redundant-load elimination, and a coalescing analysis. The GPU backends already run address-space inference, load/store vectorization, and redundant-load elimination; the project's value is the analyzer-measured before/after delta, not a claim that these transformations are new. IREE (Google, 2026) is the closest production analog for a compiler-integrated memory-optimization pipeline.

The profiler follows the activity-only tier of CUPTI and rocprofiler-sdk plus the roofline and occupancy vocabulary of Nsight Compute (NVIDIA, 2026d). It reports what it can evidence and degrades to `insufficient_evidence` when the family, PTX, or occupancy is unknown.

15 Limitations and future work

15.1 Limitations

- **No NVIDIA runtime evidence.** CUPTI capture, CUPTI profiling, and NVIDIA runtime benchmarks are compile- and unit-tested only. The rental path exists but produced no committed numbers.
- **One measured GPU.** The measured results are gfx1101 only. The analyzer supports other architectures, but their occupancy and roofline numbers are not validated here.
- **Pass deltas are projected.** The vector-width and coalescing improvements are static IR/PTX measurements. A wall-clock speedup for the transformed layernorm is not claimed.
- **The verifier is a may-analysis.** Data-dependent indices and runtime allocations degrade to non-`SAFE` verdicts; atomics, warp-synchronous code, and inter-block races

are out of scope.

- **Capture format drift.** The v1 schema is the reading contract, but the demo bundle is v0 and the current record path still uses the v0 environment-variable flow. The v1 writer path is not exercised by the demo.
- **Dashboard incompleteness.** The committed dashboard's tune section cites per-run JSONL files that are absent, and the dashboard validator is not invoked by `ci.sh`. The matmul number is quoted from the dashboard only.
- **Report schema does not enforce provenance.** The schema requires the kernel, metrics, limiter, warnings, and suggestions; the provenance object is a report-engine and dashboard convention.
- **Tooling gaps.** The auxiliary PTX dump source has no CMake target and is not built. The capture CLI's usage exit codes do not consistently match the documentation. The report names these rather than presenting a uniform interface that does not exist.
- **The archive is textual.** The consolidation removed the raw JSON/HTML evidence. The numbers in this report are preserved, but a reader who needs the raw records must regenerate them on a gfx1101 machine with the committed scripts.

15.2 Future work

1. **Run the NVIDIA rental.** Execute `rental-validate.sh` on an A100/H100 host, commit the summary with the same provenance schema, and add an NVIDIA column to the dashboard.
2. **Exercise CUDA 13.** Build the record adapter against `CUPTI_API_VERSION` 130001 and turn the Kernel9/Kernel10 `static_asserts` into a tested migration.
3. **Close the v0/v1 capture gap.** Make `record` write the v1 bundle by default, exercise the schema in the demo, and remove the environment-variable path.
4. **Strengthen the verifier.** Add HGRD-style barrier intervals, Faial-style index types, and a PTX well-formedness layer; keep the false-positive and false-negative tables current.
5. **Wire evidence into CI.** Run the dashboard validator in `ci.sh`, fix its failure path, and add a secret scan.
6. **Roofline-driven pass selection.** Choose vectorization or tiling from the profiler's bound class and the pass report's predicted delta, then validate the choice with a measured benchmark.
7. **Validate generated kernels.** Use the verifier and golden gate as a correctness harness for LLM-generated CUDA, with a kernel-benchmark sweep rather than a single family.

16 Conclusion

kernscope's claim is modest and testable: a GPU-kernel toolchain can share one library stack, one report schema, and one provenance discipline, and its tools can validate each other instead of self-reporting. The loop runs on one local GPU in 153 s. The analyzer reads the captured ISA, the verifier removes a risky variant before compilation, the capture replay is bitwise, and the pass delta is measured with the same metrics the analyzer uses. The numbers are small where the work is small (a 1.0017× capture-tune speedup) and larger where the search has room (a 1.1832× bias_gelu speedup); the report keeps them in their provenance categories.

The same discipline that makes the evidence useful makes the limits visible. There are

no NVIDIA runtime numbers, the pass deltas are projected, the verifier is not a prover, one GPU supplied every live measurement, and the dashboard is not a complete raw archive. Those are not caveats appended to the claim; they are the conditions under which the claim is true. The repository's scripts can regenerate the evidence on a gfx1101 machine, and this report is the archival account of what was present when the documentation was consolidated.

A CLI reference

Tool	Synopsis and notable options
`kernscope-analyze`	`<file.ptx|file.s|file.isa> [--arch A] [--block N] [--json|--text|--html] [--source F] [--version] [--help]`. Exit 0 clean, 1 warnings, 2 parse/arch/ schema error, 3 usage.
`kernscope-tune`	`--kernel F --shape S --arch A [--grid auto|full] [--json|--html|--text] [--db P] [--cache P] [--workdir D] [--min-occupancy X] [--no-bench] [--no-cache|--force] [--emit-repro D] [--verify] [--inject-risky F] [--gate-shape S] [--version] [--help]`. Exit 0 best selected, 1 no viable config, 2 usage/environment, 3 unknown family.
`kernscope-tune`	`--from-capture B [--vendor amd] [--arch gfx1101] [--blob-dir D] [--kernel K] [--ref F] [--candidates CSV] [--inject-config C] [--reps N] [--no-flush] [--db P] [--profile-out P] [--json|--text]`. Exit 0 best, 1 no valid config, 2 usage/environment.
`kernscope-verify`	`<src.cu|src.hip> [--device-only] [--json] [--out BASE] [--list-checks] [--budget N] [--opt O1|O2] [--arch sm_80|gfx1101] [--version]`. Exit 0 approved, 1 error-class verdict, 2 ingestion failure, 3 usage.

`kernscope-capture`	`record <app> [args] [--vendor auto|amd|nvidia] [--bundle P] [--blob-dir D] [--env K=V]`
	`replay <bundle> [--vendor A] [--arch A] [--blob-dir D] [--kernel K] [--mode bitwise|tolerance] [--ref F] [--dump-dir D] [--profile-out P]`
	`bench <bundle> [replay flags] [--reps N] [--no-flush] [--no-validate] [--db P] [--profile-out P]`
	`emit-repro <bundle> [--out D] [--blob-dir D] [--vendor A] [--arch A] [--kernel K] [--force]`
	`db-query <db.jsonl> [--arch A] [--family F] [--shape S]`. Replay exit 0 pass, 1 validation failure, 2 cannot run; db-query 0 found, 1 none, 2 unreadable.

Tool	Synopsis and notable options
`kernscope-profile`	`<binary|dir> [--vendor auto|hip|cuda] [--json|--text|--html] [--] [app args]`. Exit 0 report (including no kernels), 2 target/tracer/ schema failure, 3 usage.
`kernscope-pass-report`	`--report-pipeline gemm|layernorm|softmax [--build-dir D] [--workdir D] [--json] [--out P] [--help] [--version]`. Exit 0 emitted, 2 pipeline/analyze failure, 3 usage.
`kernscope-bench`	`[--db P] [--reps N] [--no-flush] ...`. The peak and replay harness; default database `reports/peaks/gfx1101-bench.jsonl`.
`kernscope-dashboard`	`[--out P] [--missing SECTIONS]`. Aggregates the six module sections into one self-contained HTML page and its JSON sibling.

B Verifier verdicts and engine ids

Verdict	Class	Meaning
`SAFE`	approval	The checked property was proven for the analysed path.
`MAY_OOB`	error	A bound was not proven; an access may be out of bounds.
`SYMBOLIC_DIMS`	engine (100)	Launch dimensions are symbolic; runtime check suggested.
`BARRIER_MISMATCH`	error	Barrier counts differ between paths.
`MAY_RACE`	error	Two shared accesses may race in the same phase.
`UNKNOWN_RACE`	error	The race question could not be decided; not a proof.
`PTR_OOB`	error	Pointer arithmetic may leave the object.
`DIVERGENT_BARRIER`	error	A barrier is reachable under divergent control flow.
`MAY_UNINIT_READ`	error	A read may observe an uninitialized value.
`INVALID_ADDRSPACE_CAST`	error	An address-space cast is not legal.
`SCEV_RECONSTRUCTION_FAILED`	engine (101)	SCEV could not reconstruct a range at `-O2`.
`BUDGET_EXCEEDED`	engine (102)	The per-function explored-state cap was hit.

Table 14. The verdict vocabulary. SAFE is the only approval-class verdict; engine ids are degraded outcomes, not proofs.

C Capture bundle fields

Event types: `api`, `alloc`, `free`, `unknown-free`, `launch`, `copy`, `memset`, `stream`, `fatbin`, `cubin`, `param_info`. A copy records a kind and a `cuda_kind` (1 H2D, 2 D2H, 3 D2D). A launch records how it was issued: runtime, runtime-ex, graph, driver, or driver-ex.

Object	Required fields	Notes
manifest	9	`format_id`, `schema_version`, `tool_version`, `vendor`, `arch`, `capture_mechanism`, `timestamp`, `snapshot_policy`, `size_governance`.
size governance	4	`max_snapshot_bytes` (default 1 GiB), `total_blob_bytes`, `aborted`, `abort_reason`.
event	2+	`type` and `seq`; the type-specific shape adds launch, copy, allocation, stream, module, or parameter fields.
launch	11+	`kernel_name`, `via`, `grid[3]`, `block[3]`, `shared_mem_bytes`, `stream`, `graph_handle`, `args[]`.
argument	4	`kind` (pointer/scalar), `size`, `bytes`, `va`.
blob	7	`id`, `file`, `role`, `va`, `size`, `sha256`, `patches[]`.
patch	2	`offset` and `target_va`.

Table 15. Bundle v1 required fields. All 64-bit values are lowercase 0x-prefixed hexadecimal strings; small counters and dimensions are JSON numbers. Every blob hash is recomputed on read.

D Report schema and provenance

Field	Required	Meaning
`kernel`	yes	Object with a required `name`; optional `source_file`, `arch`, `block_size`.
`metrics`	yes	Object; every value is a number.
`limiter`	yes	Human-readable binding resource.
`warnings`	yes	Array of findings.
`suggestions`	yes	Array of findings.
finding	yes	`id`, `severity`, `evidence`; `evidence` is a string-to-string map and the `text` key is the one-line renderer summary.
`schema_version`	optional	The report engine writes `"1.0"`; the schema does not require it.
`provenance`	optional	The schema does not define it. The dashboard and report engine use `provenance.mode` in {measured, projected, no_data} and `provenance.measured` as a boolean.

Table 16. The report schema and the provenance convention that sits outside it. The distinction is deliberate: the schema validates shape, and the dashboard validator checks provenance.

E Consolidation map

This report consolidates the following material. The source files were removed from the tree after the report compiled, except for the editable diagrams.

Two kinds of file were not folded: the diagrams, which are useful as standalone editable figures, and the small test inputs that the lit and architecture tests read directly. The latter are test fixtures, not report artifacts. Everything else that described the design or reported a measurement is represented here.

Folded material	Where it lives in this report
`docs/architecture.md`	Section 2, Section 3, Section 4, and the three architecture figures.
`docs/analyzer.md`, `docs/analyzer-rules.md`	Section 5 and Section A.
`docs/tuning.md`	Section 6.
`docs/verify.md`, `docs/verify-soundness.md`	Section 7 and Section B.
`docs/capture.md`, `docs/capture-format.md`, `docs/capture-mapping.md`	Section 8 and Section C.
`docs/profile.md`	Section 10.
`docs/reproducibility.md`	Section 12.
`docs/schema/report.schema.json`, `docs/schema/capture-bundle.schema.json`	Section 11.4, Section 8, Section C, Section D.
`docs/research/*.md`	Section 14 and the bibliography.
`reports/research-report/*`	This report.
`reports/demo/*`, `reports/passes/*`, `reports/peaks/*`, `reports/dashboard.*`	The measured values in Section 13 and the tool chapters. The runtime directories are regenerated by the scripts.
`reports/rentals/*`	Section 3.3; no rental numbers were present.

Table 17. Material folded into this report. The editable Excalidraw diagrams remain in `docs/diagrams/`; the test-critical before.ll inputs and the peak JSONL are retained under `tests/` because tests read them.

F Glossary

AMDGPU
AMD's GPU instruction set. The analyzer reads its assembly form; the capture path records its code objects.

bf16
16-bit bfloat16. The golden harness compares outputs in bf16 under a cosine, relative-error, and PCC contract.

coalescing
Whether the threads of a warp touch contiguous memory sectors. The analyzer uses 128-byte sectors.

CUPTI
NVIDIA's CUDA profiling tools interface. Activity records are the profiler and capture input on the NVIDIA path.

DFA / SCEV
LLVM's scalar evolution analysis; the verifier and pass report use its ranges.

gfx1101
The architecture code for the local Radeon RX 7800 XT (RDNA 3).

hipcc
The AMD HIP compiler driver used for the live path.

LDS
AMD local data share, the shared-memory name in the AMDGPU ISA.

LLVM IR
The compiler intermediate representation the verifier checks and the pass plugin rewrites.

occupancy
Active warps divided by the maximum active warps on an SM. The report's occupancy is theoretical.

PTX
NVIDIA's virtual instruction set; the analyzer's NVIDIA input.

provenance.mode
The per-module label `measured`, `projected`, or `no_data` used by the report engine and dashboard.

roofline
The compute and memory ceilings and the ridge point where they meet; the tuner and profiler use it to classify a bound.

VA-faithful
A snapshot that preserves the original virtual addresses and the pointer graph, with a patch table for remapping.

References

Ayesha Afzal, Ganesh Balakrishnan, and et al. LLOV: A fast static data-race checker for OpenMP programs. *ACM Transactions on Architecture and Code Optimization (TACO)*, 17(4), 2020. doi: 10.1145/3418597.

AMD. ROCm rocprofiler-sdk. https://rocm.docs.amd.com/projects/rocprofiler-sdk/, 2026. HIP_RUNTIME_API + kernel dispatch interception; correlation_id join; replay target of kernscope-capture.

Ali Bakhoda, George L. Yuan, Wilson W. L. Fung, Henry Wong, and Tor M. Aamodt. Analyzing CUDA workloads using a detailed GPU simulator. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 163–174, 2009.

Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. GPUVerify: A verifier for GPU kernels. In *Proceedings of the 27th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 113–132, 2012. doi: 10.1145/2384616.2384625.

Adam Betts, Nathan Chong, Alastair F. Donaldson, Jeroen Ketema, Shaz Qadeer, Paul Thomson, and John Wickerson. The design and implementation of a verification technique for GPU kernels. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 37(3):10:1–10:49, 2015. doi: 10.1145/2743017.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594, 2018. URL https://arxiv.org/abs/1802.04799. arXiv:1802.04799.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS 35)*, 2022. URL https://arxiv.org/abs/2205.14135. arXiv:2205.14135.

Google. IREE: Intermediate representation execution environment (GPUPromoteMatmulOperands, GPUReduceBankConflicts, GPUConvertToCoalescedDMA). https://github.com/iree-org/iree, 2026. MLIR-based production analog for shared-memory promotion and coalescing passes.

HGRD authors. Towards an accurate GPU data race detector. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.02106. arXiv:2604.02106.

Tyler Jacobson, Martin Burtscher, and Ganesh Gopalakrishnan. HiRace: A data-race detector for GPU kernels. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2401.04701. arXiv:2401.04701.

Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. Accel-Sim: An extensible simulation framework for validated GPU modeling. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 473–486, 2020.

Khronos Group. SPIR-V specification. https://registry.khronos.org/SPIR-V/, 2026. Universal limits §2.17; spirv-val as the well-formedness gate.

Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the 2004 International Symposium on*

Code Generation and Optimization (CGO), pages 75–86, 2004. doi: 10.1109/CGO.2004.1281665.

Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021. doi: 10.1109/CGO51591.2021.9370308.

Guodong Li and Ganesh Gopalakrishnan. Scalable SMT-based verification of GPU kernel functions. In *Proceedings of the 18th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 187–196, 2010. doi: 10.1145/1882291.1882320.

Guodong Li, Peng Li, Geoffrey Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. GKLEE: Concolic verification and test generation for GPUs. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 215–224, 2012.

Peng Li, Guodong Li, and Ganesh Gopalakrishnan. Practical symbolic race checking of GPU programs. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2014. doi: 10.1109/SC.2014.20.

Daniel Liew, Haniel Barbosa, Tiago Cogumbreiro, and et al. Sound and partially-complete static analysis of data-races in GPU programs (FaialAA). In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2024. doi: 10.1145/3689797.

Linux kernel. Linux kernel eBPF verifier. https://github.com/torvalds/linux/tree/master/kernel/bpf/verifier.c, 2026. Tnum + register-bound analysis, check_map_access(), path-sensitive pruning; the engineering template for kernscope-verify.

LLNL. Record-remix-replay (R^3): Llm evolutionary search and bayesian optimization over source, compiler, and launch configuration on a record-replay engine. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.11109. arXiv:2604.11109.

NVIDIA. CUDA C++ programming guide. https://docs.nvidia.com/cuda/cuda-programming-guide/, 2026a. v13.3; occupancy model and launch configuration worked examples (§2.3.7).

NVIDIA. CUDA profiling tools interface (CUPTI). https://docs.nvidia.com/cupti/, 2026b. Activity API; Kernel9 vs Kernel10 record kinds; Event/Metric APIs removed in CUDA 13.

NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines and solvers (incl. CuTe layout library). https://github.com/NVIDIA/cutlass, 2026c.

NVIDIA. NVIDIA Nsight compute profiling guide – speed of light (SOL) analysis. https://docs.nvidia.com/nsight-compute/ProfilingGuide/, 2026d. Sections and rules architecture: SpeedOfLight, Occupancy, SchedulerStats, WarpStateStats.

Konstantinos Parasyris, Giorgis Georgakoudis, Enrique Rangel, Ignacio Laguna, and Johannes Doerfert. Scalable tuning of (OpenMP) GPU applications via kernel record and replay. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.

Pável Pereira, Higo Albuquerque, Isabela da Silva, Hendrio Marques, Felipe Monteiro, Ricardo Ferreira, and Lucas Cordeiro. SMT-based context-bounded model checking for CUDA programs (ESBMC-GPU). *Concurrency and Computation: Practice and Experience*, 29(22), 2017. doi: 10.1016/j.scico.2017.09.005.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 519–530, 2013. doi: 10.1145/2491956.2462176.

Cole Ramos and Keith Lowery. Kerncap: Automated kernel extraction and isolation for AMD GPUs. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2605.03208. arXiv:2605.03208 (AMD Research, IntelliKit).

Konstantin Serebryany and Evgenii Stepanov. MemorySanitizer: Fast detector of uninitialized memory use in C++. In *Proceedings of the 2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 46–55, 2015.

Tally authors. Tally: Transparent virtualization layer intercepting kernel-launch APIs with PTX rewriting. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2410.07381. arXiv:2410.07381.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

Ben van Werkhoven. Kernel tuner: A search-optimizing GPU kernel auto-tuner (HIP extension). *arXiv preprint*, 2024. URL https://arxiv.org/abs/2407.11488. arXiv:2407.11488.

N. Vijaykumar and et al. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

Vasily Volkov. Better performance at lower occupancy. In *GPU Technology Conference (GTC)*, 2010.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4): 65–76, 2009. doi: 10.1145/1498765.1498785.

Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879, 2020. URL https://arxiv.org/abs/2006.06762. arXiv:2006.06762.